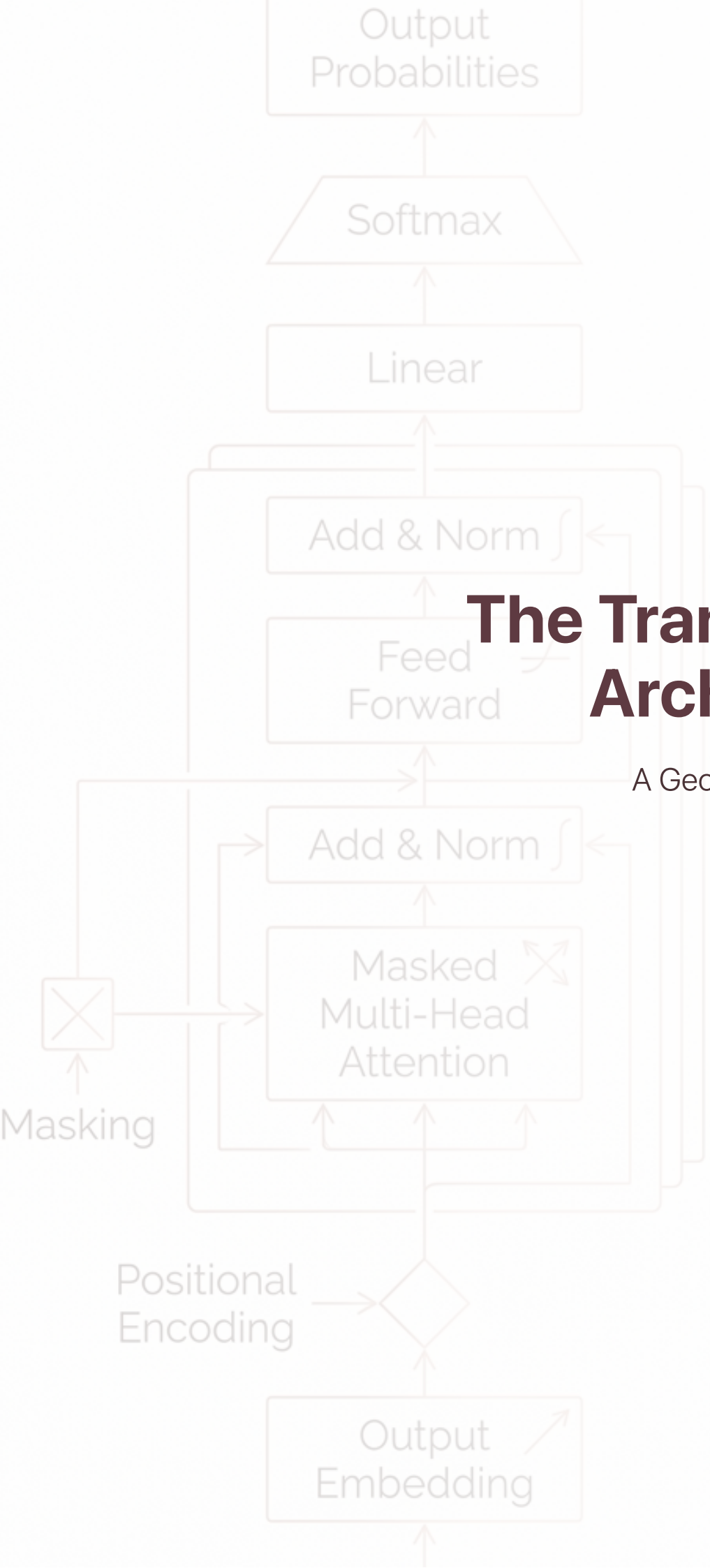


The Transformer Architecture

A Geometric Toy Example from Scratch

By Lydia Pedersen

The Transformer Architecture: A Geometric Toy Example from Scratch

Copyright © 2026 Lydia Pedersen. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Table of Contents

Preface: The Big Picture & Tensor Notation

Chapter 1: Tokens, One-Hot Encodings, and the Embedding Matrix

Chapter 2: The Permutation Invariance Problem & Positional Encoding

Chapter 3: The Motivation for Q, K, and V (Asymmetric Similarity)

Chapter 4: The Attention Score and $\sqrt{d_k}$

Chapter 5: Causal Masking

Chapter 6: From Scores to Synthesis: Softmax and The Value Matrix

Chapter 7: The Projection Matrix and The Cross-Head Mixer

Chapter 8: The Residual Stream and the Central Memory Bus

Chapter 9: Taming the Stream: The Geometry of Layer Normalization

Chapter 10: The Residual Stream and MLP Expansion

Chapter 11: Layer Normalization and The Multi-Layer Perceptron

Chapter 12: The MLP as a Key-Value Memory and Layer 1 Completion

Chapter 13: The MLP Activation and Layer 2 Self-Attention

Chapter 14: Final Layer 1 Residuals, Norms, and Layer 2 Attention

Chapter 15: Layer 2 Self-Attention

Chapter 16: Deepening the Representation: MLP and Residuals in Layer 2

Chapter 17: Mapping Back to Words

Chapter 18: Final Softmax and Predictions

Chapter 19: The Cross-Entropy Loss Function

Chapter 20: The Beautiful Cancellation

Chapter 21: Backpropagating Through the Unembedding and Residual Stream

Chapter 22: Backpropagation - Attention (Chapter 1: Softmax & Scores)

Chapter 23: Backpropagation - Attention (Chapter 2: Q, K, V)

Chapter 24: Updating the Embeddings and Conclusion

Preface: The Big Picture & Tensor Notation

The Problem with Tutorials

The typical approach to understanding the Transformer architecture presents a frustrating hurdle. Existing literature relies heavily on abstract analogies. Texts often describe the Query matrix as asking a question and the Key matrix as holding the answer. These metaphors might give a fleeting sense of intuition. They collapse the moment a practitioner attempts to write code, debug a model, or understand the fundamental geometry of deep learning.

This text takes a different approach. A complete Transformer is built from scratch by calculating every single number by hand. The goal is to translate these complex, abstract operations into meaningful, structural realities. The result is a clear, intuitive grasp of how multi-dimensional tensors flow through linear projections, removing any sense of hidden mechanics.

To accomplish this, a problem is selected that is small enough to compute on a whiteboard, yet complex enough to demonstrate the true power of the architecture.

The Toy Example

The objective is to train a Transformer to predict the next word in a specific sequence.

Input: `<BOS>` `i` `woke` `up`
Target: `late`

The token `<BOS>` stands for Beginning of Sequence and serves as a standard boundary marker indicating the start of a sequence.

This four-token prompt requires the attention mechanism to perform observable contextual work. Predicting the target token `late` requires more than inspecting the final token `up` in isolation; the network must combine `woke` and `up` into a unified phrasal concept.

The Vocabulary Space

To keep every matrix operation tractable by hand, the model operates on a fixed vocabulary of twelve tokens. The variable V denotes the total vocabulary size, setting $V = 12$ for this configuration.

`<BOS>` `<EOS>` `<PAD>` `i` `we` `woke` `stayed` `up` `late` `early` `today` `yesterday`

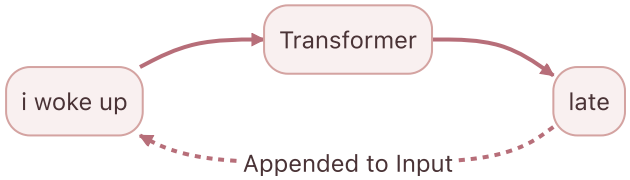
This compact vocabulary contains distinct semantic clusters. The pronouns `i` and `we` form one grammatical group, while the temporal modifiers `late`, `early`, `today`, and `yesterday` form another. These groupings allow the matrix operations to cluster related concepts geometrically in vector space during training.

The Architecture

Before specifying the tensor dimensions, the network topology must be established. The model uses an autoregressive, Decoder-only Transformer architecture, which forms the standard structural foundation for causal language modeling.

Two architectural properties define this topology:

- **Autoregressive Generation:** The network predicts the next token from its own preceding outputs. Completing a token prediction appends that newly generated token to the input sequence, and the forward pass repeats in a continuous feedback loop to generate subsequent text.
- **Decoder-Only Topology:** Early sequence-to-sequence Transformers paired a bidirectional Encoder for processing a source sequence with a causal Decoder for generating a target sequence. Next-token prediction operates on a single continuous sequence rather than translating between two distinct sequences, eliminating the Encoder and retaining only the causal Decoder stack.

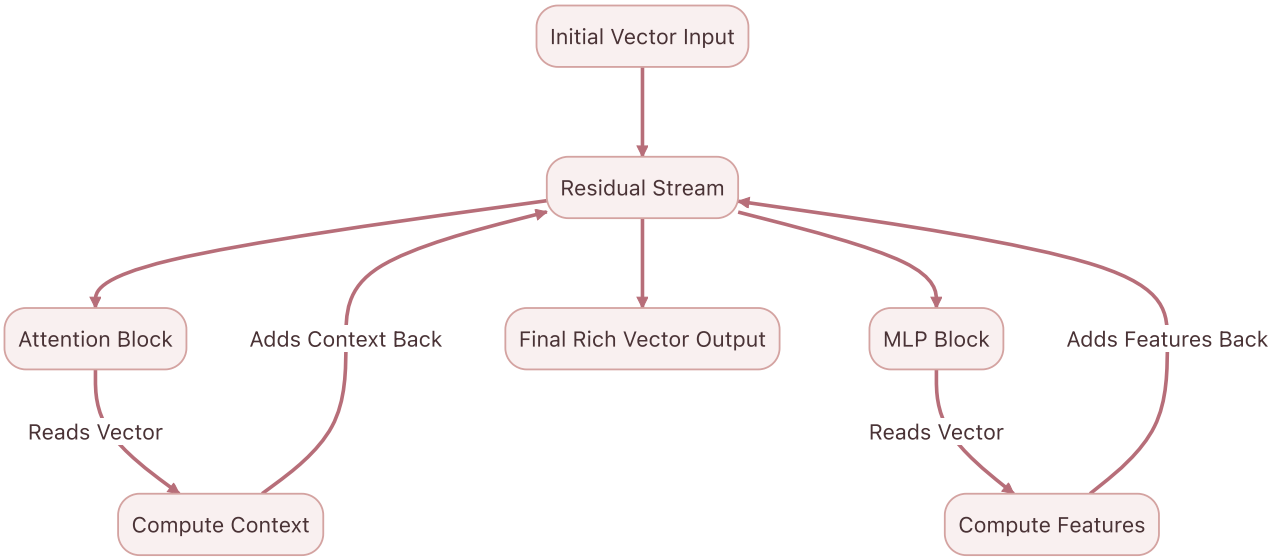


The Residual Stream

Inside this Decoder exists a central memory bus known as the residual stream, which forms the primary structural backbone of the architecture.

The residual stream acts as a continuous data highway running from the initial embedding layer to the final output projection. Entering tokens are placed onto this highway as dense vectors. Rather than overwriting those vectors in place, the Attention and Multi-Layer Perceptron (MLP) sub-layers read from the stream, compute contextual updates, and add those updates back into the original vectors.

This additive update rule preserves the initial representation without compressing it through a destructive bottleneck. Each token vector accumulates semantic and syntactic context as the forward pass progresses across layers.



The Dimensions

Six hyperparameters govern the shape of every tensor and weight matrix in the toy architecture:

Hyperparameter	Symbol	Toy Value	Architectural Role
Sequence Length	`S`	`4`	Number of tokens processed simultaneously in one forward pass
Model Dimensionality	`d_model`	`6`	Width of each token vector traveling along the residual stream
Batch Size	`B`	`1`	Number of independent token sequences processed in parallel
Attention Heads	`H`	`3`	Number of parallel subspaces used to route token-to-token context
Head Dimensionality	`d_k`, `d_v`	`2`	Subspace width per attention head (`d_model / H = 6 / 3`)
Feed-Forward Dimension	`d_ff`	`24`	Expanded hidden width of the MLP block (`4 * d_model = 24`)

Three structural tradeoffs motivate these specific hyperparameter choices:

- **Single-Sequence Batching (`B = 1`):** Parallel hardware accelerators stack multiple independent sequences into a batch to execute matrix multiplications concurrently across inputs. Setting the batch size to `1` isolates the geometry of a single four-token prompt without altering the underlying linear algebra.
- **Multi-Head Subspace Splitting (`H = 3`, `d_k = 2`):** Dividing the 6-dimensional residual stream across 3 parallel heads assigns each head a 6×2 projection matrix. Independent heads can specialize in distinct linguistic relationships simultaneously, such as pairing subjects with verbs in one subspace while tracking temporal modifiers in another.
- **Feed-Forward Expansion (`d_ff = 24`):** A 6-dimensional residual stream suffices to transport token representations between layers, whereas non-linear feature extraction requires a wider workspace. The MLP projects each 6-dimensional vector into a 24-dimensional intermediate space (following the standard $4 \times d_{\text{model}}$ expansion ratio) so entangled concepts can be linearly separated before projecting back into the residual stream.

The Central Tensor

Before text enters the Transformer, the characters must be converted into numbers. The process begins by assigning a unique integer index to every word in the vocabulary.

That integer is then mapped to a one-hot encoded vector. A one-hot vector is an array of zeros with a single `1` placed at the index corresponding to that word. For a twelve-word vocabulary, the one-hot vector for the fourth word is a 12-dimensional array consisting of eleven zeros and one `1` in the fourth position.

Projecting each one-hot vector into a continuous vector space produces the foundational tensor that travels through the network. The full three-dimensional tensor has shape $\text{Batch} \times \text{Sequence Length} \times \text{Model Dimension}$.

Under the toy hyperparameters, this tensor has shape $1 \times 4 \times 6$. Dropping the unit batch dimension ($B = 1$) leaves a 4×6 matrix X traveling along the residual stream:

$$X = \begin{bmatrix} \text{-- } \langle \text{BOS} \rangle \text{ vector --} \\ \text{-- ”i” vector --} \\ \text{-- ”woke” vector --} \\ \text{-- ”up” vector --} \end{bmatrix}_{4 \times 6}$$

Every single mathematical operation in the forward pass reads from and writes to this 4×6 matrix.

The raw text is then mapped to the vocabulary indices, preparing for the first genuine calculation. The one-hot encoded vectors are transformed into this 4×6 geometric representation.

Chapter 1: Tokens, One-Hot Encodings, and the Embedding Matrix

The Preface established that every operation in the Transformer reads from and writes to a central 4×6 matrix. Bridging the gap between raw text and that geometric representation is the next requirement. Text is inherently abstract. Computers cannot multiply words. Computers multiply numbers. A rigorous mechanical process is needed to translate human language into a mathematical format that a neural network can manipulate.

This translation executes in three sequential stages:

1. **Tokenization:** Segment the input string into discrete vocabulary symbols and map each symbol to its integer index.
2. **One-Hot Encoding:** Convert each integer index into an orthogonal unit vector in $\mathbb{R}^V$ to remove arbitrary index magnitude.
3. **Embedding Projection:** Multiply the one-hot matrix by a learned embedding matrix W_E to project the isolated basis vectors into a continuous semantic subspace $\mathbb{R}^{d_{\text{model}}}$.

The Vocabulary Space and Tokenization

The objective is to process the sequence `<BOS>` `i` `woke` `up`.

Before processing this sequence, the model requires a predefined universe of discrete symbols known as the vocabulary. In this configuration, the vocabulary consists of twelve tokens indexed from `0` to `11`:

`<BOS>` `<EOS>` `<PAD>` `i` `we` `woke` `stayed` `up` `late` `early` `today` `yesterday`

Tokenization maps each raw token in the prompt to its integer index in the vocabulary table:

- `<BOS>`: Maps to vocabulary index `0`.
- `i`: Maps to vocabulary index `3`.
- `woke`: Maps to vocabulary index `5`.
- `up`: Maps to vocabulary index `7`.

Collecting these four lookups in sequence order produces the integer index array:

$$\text{Sequence Indices} = [0, 3, 5, 7]$$

The Geometry of One-Hot Encodings

Feeding raw integer indices directly into a neural network introduces an artificial numerical hierarchy. Assigning the integer `5` to `woke` and the integer `10` to `today` causes linear layers to treat `today` as twice the scalar magnitude of `woke`. That numerical ratio is an arbitrary artifact of vocabulary ordering rather than a linguistic property.

Translating the integer indices into one-hot encoded vectors eliminates this false scalar ordering.

A one-hot vector isolates a concept geometrically. For a vocabulary of size 12, each word becomes a 12-dimensional vector filled with zeros, with a single `1.0` placed at the vocabulary index.

Converting the four tokens into one-hot vectors creates a 4×12 matrix:

$$X_{\text{one-hot}} = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & \dots & 0.0 \\ 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & \dots & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 & 0.0 & 0.0 & \dots & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 1.0 & 0.0 & 0.0 & \dots & 0.0 \end{bmatrix}$$

Treating each vocabulary entry as an independent coordinate axis in $\mathbb{R}^{12}$ places every distinct token pair at an identical Euclidean distance of $\sqrt{2}$. The resulting vectors are mutually orthogonal. The token `i` shares zero inner-product similarity with the token `woke`, replacing arbitrary index magnitude with geometric neutrality.

The Problem with Neutrality

Complete orthogonality introduces a severe representational bottleneck. The pronoun `i` shares grammatical structure with `we`, and the temporal modifier `late` shares semantic structure with `early`. Orthogonal one-hot vectors erase all pairwise similarity, forcing the network to learn syntactic rules independently for every vocabulary item.

One-hot representations also scale poorly with vocabulary size. Production language models routinely operate over vocabularies exceeding 100,000 subword tokens, making a matrix of that width computationally intractable across deep layers.

The isolated 12-dimensional vectors must be projected into a denser, lower-dimensional subspace where tokens with related meanings occupy neighboring coordinates. This dense subspace corresponds to the model dimension $d_{\text{model}} = 6$.

The Embedding Matrix

This compression is achieved using the Embedding Matrix, denoted as W_E. This matrix serves as a learnable lookup table that maps every 12-dimensional one-hot vector to a 6-dimensional coordinate vector, giving W_E a shape of 12×6.

Network initialization populates W_E with small random values. Over thousands of training iterations, backpropagation updates these coordinates and pulls semantically related tokens into neighboring regions of the vector space to minimize prediction error.

To demonstrate these calculations manually, the training phase is bypassed and a 12×6 embedding matrix is explicitly designed to show these semantic clusters:

$$W_E = \begin{bmatrix} 0.1 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.1 & 0.0 & 0.0 & 0.0 & 0.0 & 0.1 \\ 0.1 & 0.0 & 0.0 & 0.0 & 0.0 & -0.1 \\ 0.0 & 0.8 & -0.1 & 0.2 & 0.0 & 0.5 \\ 0.0 & 0.9 & -0.1 & 0.2 & 0.0 & 0.8 \\ 0.0 & -0.2 & 0.9 & 0.1 & -0.4 & 0.1 \\ 0.0 & -0.2 & 0.8 & 0.0 & 0.5 & -0.1 \\ 0.0 & -0.1 & 0.4 & 0.9 & -0.2 & 0.0 \\ 0.0 & 0.0 & -0.3 & -0.1 & 0.9 & -0.8 \\ 0.0 & 0.0 & -0.3 & -0.1 & 0.9 & 0.8 \\ 0.0 & 0.0 & -0.2 & 0.0 & 0.8 & 0.1 \\ 0.0 & 0.0 & -0.2 & -0.2 & 0.7 & -0.4 \end{bmatrix}$$

The pronoun rows at indices 3 and 4 (`i` and `we`) share large positive values in the second column. The temporal modifier rows at indices 8, 9, 10, and 11 (`late` , `early` , `today` , and `yesterday`) share large positive values in the fifth column.

The Embedding Matrix functions as a continuous coordinate map of meaning. Positioning related concepts in neighboring regions of the subspace allows downstream linear layers to generalize across clusters. A grammatical transformation learned for the coordinate region containing `i` transfers directly to `we` because both vectors occupy adjacent coordinates.

Generating the Central Tensor

To transform the neutral one-hot vectors into this rich geometric space, a standard matrix multiplication is performed. The 4×12 input matrix is multiplied by the 12×6 embedding matrix:

$$X = X_{\text{one-hot}} \times W_E$$



Multiplying a one-hot row vector by W_E selects the single row of the embedding matrix corresponding to the active token index.

The resulting 4×6 embedding matrix X takes the following form:

$$X = \begin{bmatrix} 0.1 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.8 & -0.1 & 0.2 & 0.0 & 0.5 \\ 0.0 & -0.2 & 0.9 & 0.1 & -0.4 & 0.1 \\ 0.0 & -0.1 & 0.4 & 0.9 & -0.2 & 0.0 \end{bmatrix}$$

- **Row 1 (`[0.1, 0.0, 0.0, 0.0, 0.0, 0.0]`):** Coordinate vector for `<BOS>` (vocabulary index `0`).
- **Row 2 (`[0.0, 0.8, -0.1, 0.2, 0.0, 0.5]`):** Coordinate vector for `i` (vocabulary index `3`).
- **Row 3 (`[0.0, -0.2, 0.9, 0.1, -0.4, 0.1]`):** Coordinate vector for `woke` (vocabulary index `5`).
- **Row 4 (`[0.0, -0.1, 0.4, 0.9, -0.2, 0.0]`):** Coordinate vector for `up` (vocabulary index `7`).

The text is now translated into the central 4×6 tensor that moves along the residual stream. The subsequent section examines a structural limitation in this representation and demonstrates why Transformers require positional encoding to represent sequential order.

Chapter 2: The Permutation Invariance Problem & Positional Encoding

The input sequence `<BOS>` `i` `woke` `up` is now transformed into a dense, continuous semantic space. Sparse 12-dimensional one-hot vectors are mathematically compressed into a 6-dimensional embedding matrix.

The current tensor representation X for the sequence takes the following form:

$$X = \begin{bmatrix} 0.1 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.8 & -0.1 & 0.2 & 0.0 & 0.5 \\ 0.0 & -0.2 & 0.9 & 0.1 & -0.4 & 0.1 \\ 0.0 & -0.1 & 0.4 & 0.9 & -0.2 & 0.0 \end{bmatrix}$$

This matrix captures the semantic meaning of the words. A structural limitation remains regarding sequential context.

The Permutation Invariance Problem

To understand the nature of this flaw, the processing method of the Attention mechanism must be anticipated. During the computation of self-attention, dot products are calculated between these row vectors to measure similarities.

Set operations and row-wise matrix multiplications exhibit permutation invariance. Shuffling the rows of X to represent the reordered sequence `woke` `i` `up` `<BOS>` produces identical pairwise dot products inside self-attention. The network treats `i` `woke` `up` and `woke` `i` `up` as interchangeable multisets of vectors. Natural language depends on sequential order to establish syntax and causality; `"The dog bit the man"` and `"The man bit the dog"` contain identical tokens while describing opposite events.

Without an explicit mechanism to encode sequence order, the Transformer collapses into a permutation-invariant bag-of-words model.

Injecting Time: Positional Encoding

Positional information must be explicitly injected into the vectors before they enter the attention layers. This injection is achieved by creating a secondary matrix of identical dimensions to the input tensor and adding the two matrices element by element.

Two primary design paradigms encode sequence order in Transformer architectures:

- **Relative Positional Encoding:** The attention computation encodes pairwise token distances rather than fixed sequence indices. Instead of assigning `woke` a static index of `2`, the mechanism records that `woke` sits one step after `i`, using pairwise distance biases or rotary coordinate transformations.
- **Absolute Positional Encoding:** Every sequence index receives a unique, deterministic vector signature added directly to the token embedding. Position `0` applies one fixed geometric translation in $\mathbb{R}^{d_{\text{model}}}$, position `1` applies a second translation, and each subsequent index shifts its token vector along a distinct trajectory.

This walkthrough uses a deterministic absolute positional encoding with entries bounded in $[-1, 1]$ so the variance of the token embeddings remains controlled.

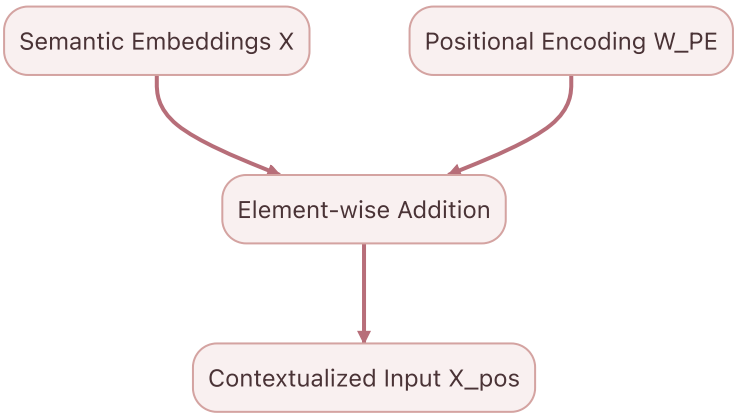
Sinusoidal positional encoding pairs sine and cosine functions of geometrically increasing wavelengths across the $d_{\text{model}} = 6$ dimensions. Using a compact base wavelength of 10 suited to a 6-dimensional toy space, each coordinate pair $(2i, 2i+1)$ for $i \in \{0, 1, 2\}$ at sequence index $pos \in \{0, 1, 2, 3\}$ follows $\text{PE}_{(pos,2i)} = \sin(pos/10^{2i/6})$ and $\text{PE}_{(pos,2i+1)} = \cos(pos/10^{2i/6})$. Rounding this wave grid to one decimal place for manual arithmetic yields the 4×6 Positional Encoding matrix W_{PE}:

$$W_{PE} = \begin{bmatrix} 0.0 & 1.0 & 0.0 & 1.0 & 0.0 & 1.0 \\ 0.8 & 0.5 & 0.4 & 0.9 & 0.2 & 1.0 \\ 0.9 & -0.4 & 0.8 & 0.6 & 0.4 & 0.9 \\ 0.1 & -1.0 & 1.0 & 0.2 & 0.6 & 0.8 \end{bmatrix}$$

This matrix exhibits geometric regularity. The values fluctuate smoothly between `-1.0` and `1.0`. Position `0` produces an alternating `[0.0, 1.0, 0.0, 1.0, 0.0, 1.0]` pattern, while subsequent positions introduce distinct phase shifts across the three frequency pairs. No two rows are identical.

The Final Matrix Addition

An element-wise matrix addition merges the semantic embeddings X and the positional signatures W_{PE}:



Adding the embedding matrix X and the positional matrix W_{PE} element by element produces the positionally encoded input tensor:

$$X_{pos} = \begin{bmatrix} 0.1 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.8 & -0.1 & 0.2 & 0.0 & 0.5 \\ 0.0 & -0.2 & 0.9 & 0.1 & -0.4 & 0.1 \\ 0.0 & -0.1 & 0.4 & 0.9 & -0.2 & 0.0 \end{bmatrix} + \begin{bmatrix} 0.0 & 1.0 & 0.0 & 1.0 & 0.0 & 1.0 \\ 0.8 & 0.5 & 0.4 & 0.9 & 0.2 & 1.0 \\ 0.9 & -0.4 & 0.8 & 0.6 & 0.4 & 0.9 \\ 0.1 & -1.0 & 1.0 & 0.2 & 0.6 & 0.8 \end{bmatrix}$$

This operation yields the final, positionally aware tensor X_{pos} :

$$X_{pos} = \begin{bmatrix} 0.1 & 1.0 & 0.0 & 1.0 & 0.0 & 1.0 \\ 0.8 & 1.3 & 0.3 & 1.1 & 0.2 & 1.5 \\ 0.9 & -0.6 & 1.7 & 0.7 & 0.0 & 1.0 \\ 0.1 & -1.1 & 1.4 & 1.1 & 0.4 & 0.8 \end{bmatrix}$$

The third row vector no longer represents the isolated lexical concept `woke` in a vacuum; its coordinates now encode `woke` anchored at sequence index `2`.

The initial input preparations are complete. A sequence of discrete tokens has been mapped into a continuous 4×6 tensor that encodes both lexical identity and sequential position, ready to enter Layer 1 Self-Attention.

Chapter 3: The Motivation for Q, K, and V (Asymmetric Similarity)

The permutation invariance problem is solved by adding absolute positional encodings to the token embeddings. The sequence `<BOS>` `i` `woke` `up` is now represented by the 4×6 matrix X_{pos} , which contains both semantic meaning and positional context.

The next step in the Transformer architecture is self-attention. The core mechanism of attention involves discovering which tokens in the sequence are relevant to each other. The simplest way to measure mathematical relevance between two vectors is to calculate their dot product. A naive assumption suggests computing the dot product of every token vector with every other token vector directly.

This approach is known as computing symmetric similarity. The process involves multiplying X_{pos} by its own transpose X_{pos}^T :

$$\text{Symmetric Similarity} = X_{pos} \times X_{pos}^T$$

Evaluating that product produces the 4×4 symmetric similarity matrix:

$$\text{Symmetric Similarity} = \begin{bmatrix} 3.01 & 3.98 & 1.19 & 0.81 \\ 3.98 & 5.92 & 2.72 & 1.56 \\ 1.19 & 2.72 & 5.55 & 4.70 \\ 0.81 & 1.56 & 4.70 & 5.19 \end{bmatrix}$$

The dot product measures directional alignment between vectors. Multiplying any real matrix by its own transpose concentrates the largest magnitudes along the main diagonal. The token `woke` aligns most strongly with itself at a score of 5.55 , and the token `up` aligns most strongly with itself at 5.19 .

This diagonal dominance exposes a structural limitation of raw inner products. Linguistic relationships are inherently asymmetric: verbs govern subjects, and particles modify verbs. Interpreting the sequence requires the particle `up` to attend backward to `woke` to resolve the phrasal verb `woke up` . Symmetric similarity forces every token vector to match most strongly with its own coordinate direction, preventing tokens from selectively targeting complementary parts of speech that occupy different regions of the embedding space.

A mechanism is required to allow a token to issue a targeted query while permitting other tokens to advertise complementary keys. Asymmetric similarity fulfills this requirement.

The Bilinear Form

To achieve asymmetric similarity, the Transformer projects the input tensor X_{pos} into three distinct subspaces using three learned weight matrices: Queries, Keys, and Values. The immediate focus remains on the Queries and Keys.

Instead of measuring the direct inner product between vector A and vector B , vector A is projected into a Query subspace and vector B is projected into a Key subspace. Alignment is then measured between the Query projection of A and the Key projection of B .

Mathematically, this operation defines a bilinear form. Given the input X_{pos} and two weight matrices W_Q and W_K, the attention scores are calculated as follows:

$$\text{Attention Scores} = (X_{pos}W_Q)(X_{pos}W_K)^T$$

The learned projection matrices W_Q and W_K act as asymmetric linear lenses. During training, backpropagation adjusts these matrices to project grammatically complementary tokens into aligned regions of a shared lower-dimensional subspace. The Query projection of a particle such as `up` and the Key projection of a verb such as `woke` can map to nearly collinear coordinates in $\mathbb{R}^{d_k}$, producing a large positive dot product that routes attention across the pair.

Calculating Queries and Keys

The Transformer uses 3 attention heads across a model dimension of $d_{model} = 6$. Dividing the model dimension by the number of heads sets the Query and Key subspace dimension to $d_k = 2$.

Each attention head maintains independent W_Q and W_K projection matrices of shape 6×2. Dedicating separate subspace projections to each head enables the network to track multiple linguistic relationships in parallel: Head 1 can align subject-verb pairs while Head 2 tracks temporal modifiers.

A concrete W_Q and W_K for the first attention head can be instantiated as follows:

$$W_Q = \begin{bmatrix} 0.1 & 0.2 \\ -0.1 & 0.5 \\ 0.8 & -0.2 \\ 0.3 & 0.4 \\ -0.2 & 0.1 \\ 0.1 & -0.3 \end{bmatrix}$$

$$W_K = \begin{bmatrix} -0.2 & 0.4 \\ 0.5 & -0.1 \\ 0.6 & 0.2 \\ 0.1 & 0.7 \\ 0.2 & -0.2 \\ -0.4 & 0.3 \end{bmatrix}$$

To calculate the Queries Q, the positionally encoded sequence X_{pos} is multiplied by W_Q:

$$Q = X_{pos} \times W_Q$$

The resulting Query matrix Q has dimensions 4×2, compressing each token from a 6-dimensional representation into a 2-dimensional Query vector:

$$Q = \begin{bmatrix} 0.31 & 0.62 \\ 0.63 & 0.76 \\ 1.82 & -0.48 \\ 1.57 & -0.57 \end{bmatrix}$$

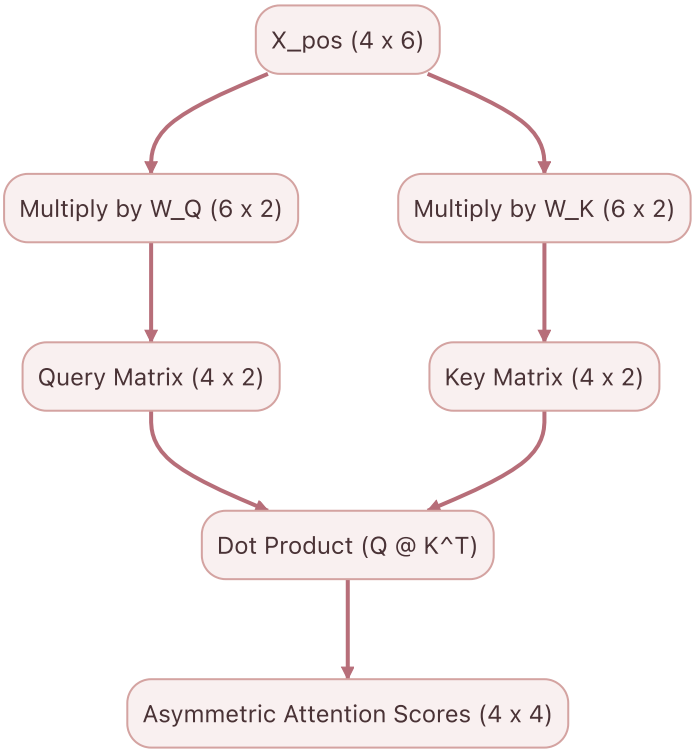
Multiplying X_{pos} by the Key projection matrix W_K projects the sequence into the Key subspace:

$$K = X_{pos} \times W_K$$

The multiplication produces the 4×2 Key matrix K , assigning each token a 2-dimensional Key vector:

$$K = \begin{bmatrix} 0.18 & 0.94 \\ 0.22 & 1.43 \\ 0.21 & 1.55 \\ 0.14 & 1.36 \end{bmatrix}$$

Separating the input into independent Query and Key projections breaks the symmetry of raw inner products. The token `woke` no longer defaults to attending primarily to itself; it projects a targeted search vector into the Query subspace and a distinct descriptor vector into the Key subspace. Subsequent operations multiply these projected matrices to compute the pairwise attention scores and scale the resulting variance to prevent gradient saturation.



Chapter 4: The Attention Score and √d_k

The previous section established why the Transformer does not calculate attention directly from the input embeddings. The sequence is projected into two distinct semantic subspaces, yielding a matrix of Queries (Q) and a matrix of Keys (K). This asymmetric projection allows the network to match concepts that belong together even if their base embeddings are geometrically distant.

The sequence currently consists of four tokens:

`<BOS>` `i` `woke` `up`

Computing the pairwise attention scores quantifies how strongly each token in the sequence attends to every other token. Taking the dot product between every Query vector and every Key vector produces this alignment matrix.

The Dot Product as a Metric of Similarity

The dot product measures geometric alignment between two vectors. Vectors pointing in similar directions yield a large positive product, orthogonal vectors yield zero, and vectors pointing in opposite directions yield a negative product.

Multiplying the Query matrix by the transpose of the Key matrix ($Q \times K^T$) computes the dot product for every possible pair of tokens in a single operation.

The specific matrices for Head 1 of the network are as follows:

$$Q = \begin{bmatrix} 0.31 & 0.62 \\ 0.63 & 0.76 \\ 1.82 & -0.48 \\ 1.57 & -0.57 \end{bmatrix}$$

$$K^T = \begin{bmatrix} 0.18 & 0.22 & 0.21 & 0.14 \\ 0.94 & 1.43 & 1.55 & 1.36 \end{bmatrix}$$

The multiplication yields the unscaled attention scores:

$$Q \times K^T = \begin{bmatrix} 0.64 & 0.95 & 1.03 & 0.89 \\ 0.83 & 1.23 & 1.31 & 1.12 \\ -0.12 & -0.29 & -0.36 & -0.40 \\ -0.25 & -0.47 & -0.55 & -0.56 \end{bmatrix}$$

Each row in this matrix corresponds to a Query token, and each column corresponds to a Key token. The entry at row 3, column 2 (-0.29) represents the unscaled alignment score between the Query vector for `woke` and the Key vector for `i`.

The Problem of Dimensionality

These raw scores are mathematically valid, yet they cannot be passed directly into normalization without rescaling. The Transformer architecture converts raw alignment scores into a probability distribution using the Softmax function, which forces the entries in each row to sum to 1.0 .

Dot products carry a dimensional scaling hazard. Increasing the vector dimension d_k causes the variance of the inner product to grow proportionally with d_k .

Two independent random vectors of dimension d_k with zero mean and unit variance produce a dot product with zero mean and variance d_k . The toy architecture uses a narrow head dimension of $d_k = 2$, keeping this variance growth small. Large-scale Transformer architectures commonly configure head dimensions of $d_k = 64$ or $d_k = 128$, inflating the variance of unscaled dot products by two orders of magnitude.

Softmax Saturation and Gradient Death

Understanding why high variance is fatal requires examining how the Softmax function behaves with extreme values.

A scenario involving a head dimension of 512 causes the variance of the dot products to hover around 512 . A single row of the unscaled attention scores might look like this:

```
[ 11.24, -3.13, 14.66, 34.46 ]
```

Applying the Softmax function to these numbers heavily amplifies the largest value through exponentiation. The resulting probability distribution becomes sharply peaked:

```
[ 0.00, 0.00, 0.00, 1.00 ]
```

The network places 100% of its attention on the final token. This one-hot concentration appears decisive at inference time, yet it causes a severe failure mode during optimization.

Neural networks update parameters via backpropagation, which scales weight updates by the local gradient of each operation. A sharply peaked Softmax distribution pushes the function into its asymptotic flat regions where the local derivative approaches zero. Vanishing gradients prevent weight updates from propagating backward through the attention projections, halting training through a failure mode known as Softmax saturation.

The Mathematical Solution: Scaling by $\sqrt{d_k}$

Stabilizing the attention logits across arbitrary head dimensions requires dividing the unscaled dot products by the square root of the head dimension ($\sqrt{d_k}$).

Dividing a random variable by a scalar constant c divides its variance by c^2 . Scaling the dot products by $1/\sqrt{d_k}$ divides the raw variance d_k by $(\sqrt{d_k})^2 = d_k$, restoring a unit variance of 1 regardless of subspace width.

This scaling factor can be applied to the synthetic large-dimension example. Dividing the raw values by $\sqrt{512}$ yields a much tighter range:

```
[ 0.50, -0.14, 0.65, 1.52 ]
```

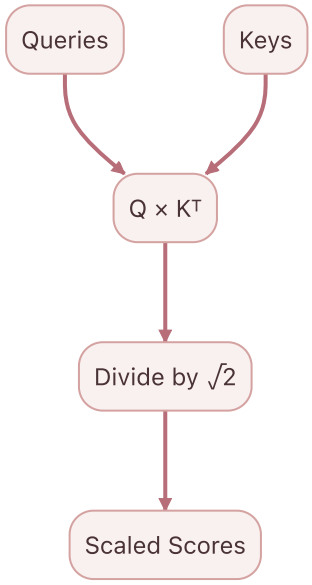
Passing these scaled numbers through the Softmax function produces a healthy, nuanced probability distribution:

```
[ 0.18, 0.10, 0.21, 0.51 ]
```

The gradients can flow freely through this distribution. The network can continue to learn.

Scaling the Toy Model

This mandatory scaling must now be applied to the toy model. The head dimension is $d_k = 2$, making the scaling factor $\sqrt{2} \approx 1.4142$.



Dividing the unrounded QK^T dot products by $\sqrt{2} \approx 1.4142$ and rounding to two decimal places yields the scaled score matrix:

$$\text{Scaled Scores} = \begin{bmatrix} 0.45 & 0.68 & 0.73 & 0.63 \\ 0.59 & 0.87 & 0.93 & 0.79 \\ -0.09 & -0.20 & -0.26 & -0.28 \\ -0.18 & -0.33 & -0.39 & -0.39 \end{bmatrix}$$

These values are now safely bounded and ready to be converted into probabilities.

The Softmax function cannot be applied immediately. The model is currently observing the entire sequence simultaneously. The first token (`<BOS>`) has a score of `0.63` connecting it to the future token `up`. In a language modeling task, allowing a token to attend to words that have not been generated yet is invalid. The future must be hidden before the probabilities are finalized. This requirement introduces the mathematics of Causal Masking.

Chapter 5: Causal Masking

The scaled attention scores have now been derived. The dot product between the Query and Key matrices was calculated to measure how strongly each token seeks information from every other token, and the result was scaled by $\sqrt{d_k}$ to prevent gradient saturation.

Before converting these scores into a final probability distribution, a structural constraint governing parallel training must be enforced.

The Problem of Parallel Training

During Transformer training, tokens are not processed sequentially one step at a time. Parallel training passes the entire target sequence through the network simultaneously using a technique known as teacher forcing. Matrix multiplications compute the attention scores for `<BOS>`, `i`, `woke`, and `up` concurrently in a single forward pass.

Examining the scaled attention scores from the previous calculation illustrates this behavior. The rows represent the Queries seeking information, and the columns represent the Keys offering information:

$$\text{Scaled Scores} = \begin{bmatrix} 0.45 & 0.68 & 0.73 & 0.63 \\ 0.59 & 0.87 & 0.93 & 0.79 \\ -0.09 & -0.20 & -0.26 & -0.28 \\ -0.18 & -0.33 & -0.39 & -0.39 \end{bmatrix}$$

The first row represents the `<BOS>` token acting as a Query. It generates attention scores against all available Keys. The second column of the first row holds a score of `0.68`, representing the `<BOS>` token attending to the `i` token.

That non-zero weight exposes a causality violation during parallel training. Predicting the token immediately following `<BOS>` permits the model to condition only on `<BOS>` itself. In the unmasked score matrix, the first row grants `<BOS>` direct visibility into the future tokens `i`, `woke`, and `up`. Allowing past positions to read future positions leaks the target labels into the forward pass, teaching the network to copy upcoming tokens rather than modeling causal structure.

The Causal Mask

Preserving autoregressive causality requires blocking information flow from future tokens into past positions. Applying a lower-triangular causal mask to the scaled score matrix enforces this directional boundary.

A mask is defined where any position representing a Query attending to a future Key is marked for obstruction:

$$\text{Mask} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Where the mask holds a `1`, the original scaled score is retained. Where the mask holds a `0`, the score is overwritten with negative infinity ($-\infty$). Applying this operation yields the masked attention scores:

$$\text{Masked Scores} = \begin{bmatrix} 0.45 & -\infty & -\infty & -\infty \\ 0.59 & 0.87 & -\infty & -\infty \\ -0.09 & -0.20 & -0.26 & -\infty \\ -0.18 & -0.33 & -0.39 & -0.39 \end{bmatrix}$$

Inspecting the second row confirms that the Query for the token `i` can attend only to the Key for `<BOS>` and the Key for `i`. The upper-triangular scores for `woke` and `up` are masked out, preserving temporal causality.

The Mathematical Role of Negative Infinity

The value $-\infty$ is used rather than zero due to the algebraic properties of the Softmax normalization step that follows. The Softmax function converts each row of scores into a probability distribution by exponentiating every entry via e^x .

In the limit $x \to -\infty$, the exponential e^x evaluates to 0 . Normalizing the masked rows through the Softmax function assigns a probability weight of 0.00 to every upper-triangular entry, ensuring that future tokens contribute zero weight to the representation of past tokens.

With the causal mask in place, the masked scores can pass through the Softmax function to weight the Value vectors.

Chapter 6: From Scores to Synthesis: Softmax and The Value Matrix

The previously calculated masked attention scores provide a strict mathematical barrier that prevents information from flowing backward in time through the application of a lower-triangular matrix of negative infinity values. The resulting matrix represents the raw geometric alignment between Queries and Keys across all valid time steps.

These scalar values are mathematically unbounded. Converting them into a stable format capable of driving the core synthesis step of the attention mechanism requires the Softmax function and the introduction of a third fundamental learned matrix: the Value matrix.

The Softmax Function: Converting Alignment to Probability

The attention scores function as routing weights for a weighted sum across token vectors. Using raw unbounded scores directly would cause vector magnitudes to compound uncontrollably as signals propagate through deeper layers. Maintaining numerical stability requires weights that are non-negative and sum to 1.0 across each row. Applying the row-wise Softmax function enforces both constraints.

The Softmax function operates by taking the exponential of each input value and dividing it by the sum of all exponentials in that row. Exponentiation maps any real number to a positive value. Dividing by the total sum normalizes these positive values into a strict probability distribution.

The masked scaled scores from the preceding step are:

$$\text{Scores}_{masked} = \begin{bmatrix} 0.45 & -\infty & -\infty & -\infty \\ 0.59 & 0.87 & -\infty & -\infty \\ -0.09 & -0.20 & -0.26 & -\infty \\ -0.18 & -0.33 & -0.39 & -0.39 \end{bmatrix}$$

Applying the Softmax function yields the final attention weights matrix A :

$$A = \text{Softmax}(\text{Scores}_{masked}) = \begin{bmatrix} 1.00 & 0.00 & 0.00 & 0.00 \\ 0.43 & 0.57 & 0.00 & 0.00 \\ 0.37 & 0.33 & 0.31 & 0.00 \\ 0.29 & 0.25 & 0.23 & 0.23 \end{bmatrix}$$

Under the causal mask, exponentiating $-\infty$ yields zero for every upper-triangular position, converting masked future connections into zero-valued weights. The network cannot extract information from future tokens, and each row sums to 1.00 before two-decimal rounding (in Row 3, $[0.3654, 0.3258, 0.3088, 0.0000]$ rounds to $[0.37, 0.33, 0.31, 0.00]$), forming a valid probability distribution over all preceding context.

The Value Matrix: The Content Payload

Computations thus far have focused on routing. The Query and Key matrices determine where information flows by scoring pairwise relevance, whereas the Value matrix carries the content payload.

Scoring pairwise alignment and updating token representations require distinct semantic features, so the architecture projects the positionally encoded embeddings X_{pos} into a third subspace using the learned Value weight matrix W_V .

The Value projection maps the model dimension $d_{model} = 6$ down to the head value dimension $d_v = 2$ via the 6×2 weight matrix W_V :

$$W_V = \begin{bmatrix} 0.3 & -0.1 \\ 0.2 & 0.5 \\ -0.4 & 0.1 \\ 0.1 & 0.6 \\ -0.3 & 0.2 \\ 0.5 & -0.4 \end{bmatrix}$$

Multiplying the positionally encoded input matrix X_{pos} by W_V produces the 4×2 Value matrix V :

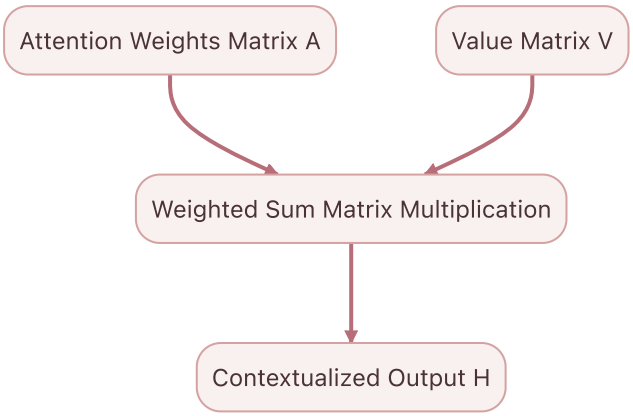
$$V = X_{pos} \cdot W_V = \begin{bmatrix} 0.83 & 0.69 \\ 1.18 & 0.70 \\ 0.04 & -0.20 \\ -0.36 & 0.00 \end{bmatrix}$$

The matrix V contains the actual conceptual representations that will be broadcast across the sequence. Each row holds the information payload for a single token in the `<BOS>` `i` `woke` `up` sequence.

The Weighted Sum: Synthesizing Context

The culmination of the single-head attention mechanism relies on a matrix of routing instructions A and a matrix of information payloads V . New contextualized representations are synthesized by computing the matrix product of A and V .

This operation executes a weighted sum across rows. Every token constructs an updated representation by blending the Value vectors of all preceding tokens according to the probabilities in the attention matrix.



Multiplying the unrounded Softmax attention weights A by V and rounding to two decimal places yields the contextualized head output H :

$$H = A \cdot V = \begin{bmatrix} 1.00 & 0.00 & 0.00 & 0.00 \\ 0.43 & 0.57 & 0.00 & 0.00 \\ 0.37 & 0.33 & 0.31 & 0.00 \\ 0.29 & 0.25 & 0.23 & 0.23 \end{bmatrix} \cdot \begin{bmatrix} 0.83 & 0.69 \\ 1.18 & 0.70 \\ 0.04 & -0.20 \\ -0.36 & 0.00 \end{bmatrix} = \begin{bmatrix} 0.83 & 0.69 \\ 1.03 & 0.70 \\ 0.70 & 0.42 \\ 0.46 & 0.32 \end{bmatrix}$$

The final row corresponding to the token `up` has a new representation of `[0.46, 0.32]`. This vector is no longer a static dictionary definition. It is a dynamic, context-aware representation shaped by the presence of `woke` and `i` occurring earlier in the sequence.

The attention mechanism for a single head is complete. The model operates with three independent attention heads running in parallel. The architecture then reconciles these independent perspectives by projecting them back into the original embedding dimension.

Chapter 7: The Projection Matrix and The Cross-Head Mixer

The computations detailed in Parts 5 and 6 resolve the core attention mechanism, yielding isolated outputs from the parallel attention heads of the network. These discrete features must now combine into a unified structure before projecting the contextual representations forward.

The Concatenation Step

The previous phase completed the journey of a single attention head. The mechanism calculated its masked attention scores, converted those scores into strict probability distributions via the Softmax function, and finally computed a weighted sum over the Value matrix V.

That process yields a contextually enriched vector for each token in the sequence. These vectors only possess a dimension of $d_v = 2$, whereas the overall model dimension is $d_{model} = 6$. The architecture deliberately splits into three parallel attention heads so the network can simultaneously look for different types of semantic relationships. Head 1 might attend to subject-verb pairings, while Head 2 looks for temporal markers, and Head 3 focuses on pronoun antecedents.

The system now faces a structural challenge. Three isolated sets of findings exist. The model must unify these independent representations back into a single cohesive vector for each token that matches the overarching $d_{model} = 6$ dimension.

The most direct way to combine the outputs of the three heads might appear to be element-wise addition. Summing the three 4×2 matrices destroys the distinct subspace features extracted by each head. A strong positive activation in Head 1 paired with a strong negative activation in Head 2 cancels out under addition, erasing the directional evidence gathered by both heads.

Instead of summing, the architecture concatenates the outputs along the feature dimension. Placing the three 4×2 matrices side by side preserves every subspace coordinate. The resulting matrix has a sequence length of 4 and a combined feature dimension of $3 \times 2 = 6$.

The numerical outputs of the three heads illustrate this concatenation step. The Head 1 matrix calculated in Chapter 6 sits alongside the corresponding outputs from Head 2 and Head 3:

$$\text{Head 1} = \begin{bmatrix} 0.83 & 0.69 \\ 1.03 & 0.70 \\ 0.70 & 0.42 \\ 0.46 & 0.32 \end{bmatrix}$$

$$\text{Head 2} = \begin{bmatrix} -0.50 & 0.10 \\ -0.40 & 0.30 \\ -0.20 & 0.25 \\ 0.10 & -0.15 \end{bmatrix}$$

$$\text{Head 3} = \begin{bmatrix} 0.20 & -0.30 \\ 0.15 & -0.20 \\ 0.40 & 0.05 \\ 0.25 & 0.10 \end{bmatrix}$$

Concatenating these three matrices horizontally achieves the target width of 6:

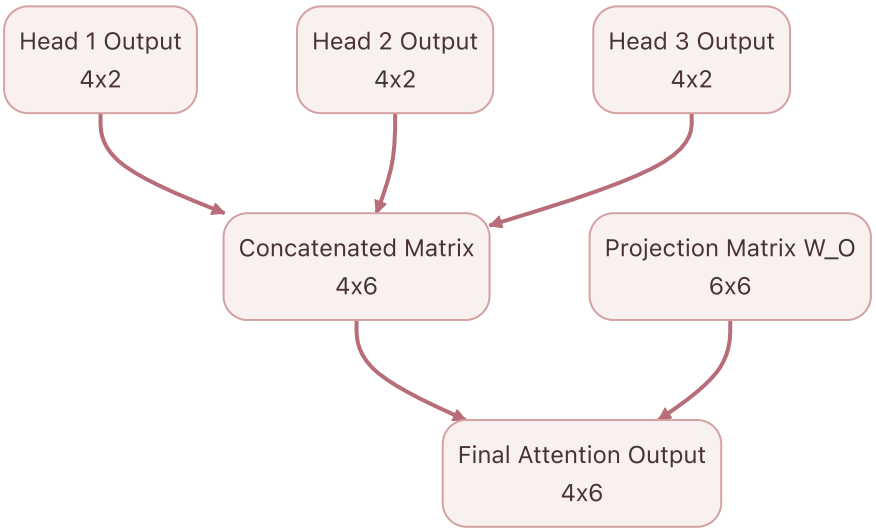
$$\text{Concatenated} = \begin{bmatrix} 0.83 & 0.69 & -0.50 & 0.10 & 0.20 & -0.30 \\ 1.03 & 0.70 & -0.40 & 0.30 & 0.15 & -0.20 \\ 0.70 & 0.42 & -0.20 & 0.25 & 0.40 & 0.05 \\ 0.46 & 0.32 & 0.10 & -0.15 & 0.25 & 0.10 \end{bmatrix}$$

The Projection Matrix

Concatenation restores the required 4×6 matrix dimensions, yet a geometric isolation problem remains. The features stay partitioned by head: columns 1 and 2 belong exclusively to Head 1, columns 3 and 4 to Head 2, and columns 5 and 6 to Head 3. The extracted features occupy the same matrix without interacting.

Synthesizing discrete subspace features into higher-order concepts requires cross-head interaction. The architecture achieves this synthesis through the final learned parameter of the attention block: the output projection matrix W_O .

Tensor	Symbol	Shape	Architectural Role
Head 1 Output	`Head 1`	`4 x 2`	Captures subspace 1 contextual payload (`Columns 1–2`)
Head 2 Output	`Head 2`	`4 x 2`	Captures subspace 2 contextual payload (`Columns 3–4`)
Head 3 Output	`Head 3`	`4 x 2`	Captures subspace 3 contextual payload (`Columns 5–6`)
Concatenated Heads	`Concatenated`	`4 x 6`	Aligns all three `4 x 2` head outputs side by side
Output Projection	`W_O`	`6 x 6`	Mixes features across all heads via learned linear weights
Attention Block Output	`Output`	`4 x 6`	Produces unified contextual update for the residual stream



The matrix W_O has dimensions of $d_{model} \times d_{model}$, which in this configuration is 6×6 . It acts as a cross-head mixer. Multiplying the concatenated matrix by W_O produces a resulting matrix that is a linear combination of all features across all heads. The network can learn that a high value in column 1 from Head 1 combined with a low value in column 5 from Head 3 encodes a composite syntactic signal that should pass forward to the rest of the architecture.

The initialized projection matrix W_O for the toy model takes the following form:

$$W_O = \begin{bmatrix} 0.10 & 0.20 & -0.10 & 0.30 & 0.00 & -0.20 \\ -0.20 & 0.10 & 0.40 & 0.10 & 0.20 & 0.00 \\ 0.30 & -0.10 & 0.10 & -0.20 & 0.50 & 0.10 \\ -0.10 & 0.40 & -0.30 & 0.10 & -0.10 & 0.30 \\ 0.20 & 0.00 & 0.20 & -0.10 & 0.30 & -0.10 \\ -0.30 & 0.10 & -0.10 & 0.20 & -0.20 & 0.40 \end{bmatrix}$$

Multiplying the concatenated outputs by W_O applies the cross-head linear transformation:

$$\text{Output} = \text{Concatenated} \cdot W_O$$

$$\text{Output} = \begin{bmatrix} -0.08 & 0.29 & 0.18 & 0.35 & 0.00 & -0.33 \\ -0.10 & 0.42 & 0.10 & 0.43 & -0.01 & -0.25 \\ -0.03 & 0.31 & 0.08 & 0.29 & 0.07 & -0.10 \\ 0.05 & 0.06 & 0.18 & 0.13 & 0.18 & -0.11 \end{bmatrix}$$

Rejoining the Stream

This calculation completes the Multi-Head Self-Attention block. The process began with token embeddings representing the sequence `<BOS>` `i` `woke` `up`. The architecture projected those representations into three subspaces, gathered causal context across the sequence, and fused the resulting head outputs into a unified 4×6 matrix.

Every row vector in this output matrix now encodes contextualized information drawn from its preceding tokens. These updates are ready to merge back into the main residual stream of the Transformer.

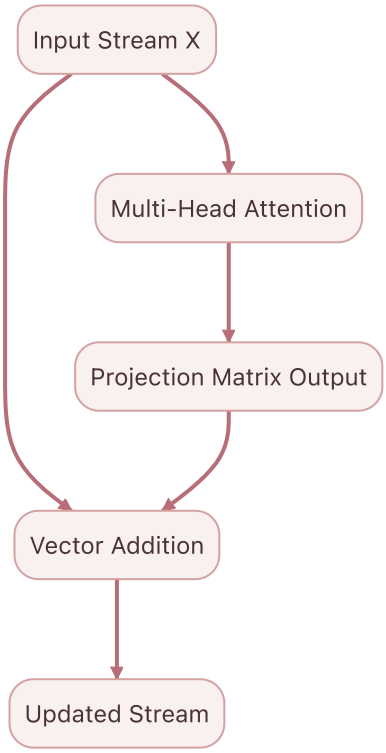
Chapter 8: The Residual Stream and the Central Memory Bus

The multi-head attention output has been calculated. Treating this output as the sole input to the next layer is a common instinct in traditional feed-forward networks. The Transformer architecture abandons this sequential pipeline. It instead relies on a central shared memory backbone known as the residual stream.

Reframing the Architecture: The Information Highway

In a traditional sequential neural network, each layer overwrites the representation completely because the input to layer two is exclusively the output of layer one. This sequential overwrite creates an information bottleneck: any feature destroyed during an intermediate transformation is lost permanently. Backpropagation through a strictly sequential chain also forces gradients to multiply through the weight matrix of every layer. Small singular values across those weight matrices cause the backpropagated gradients to vanish exponentially, halting weight updates in early layers.

The Transformer solves both problems by treating the network as a continuous highway of information rather than a sequence of overwrites. The positionally encoded input embeddings travel straight through the entire network, from the first block to the final output. The attention mechanisms and feed-forward networks sit alongside this highway. They read from the stream, perform their specialized computations, and write their results back into the stream via addition.



The token vectors do not lose their original identity. The attention block acts as an additive update, mixing contextual information into the base meaning of the token.

The Mathematics of the Residual Connection

This additive update is formalized with a direct equation:

$$X_{\text{out}} = X_{\text{in}} + \text{Attention}(X_{\text{in}})$$

Here, X_{in} is the state of the residual stream entering the attention block, and $\text{Attention}(X_{\text{in}})$ is the 4×6 output calculated in Chapter 7 using the output projection matrix W_O .

To match the two-decimal precision of $\text{Attention}(X_{\text{in}})$, the residual stream matrix X_{in} retains the two-decimal sinusoidal evaluation of $X + W_{PE}$ from Chapter 2:

$$X_{\text{in}} = \begin{bmatrix} 0.10 & 1.00 & 0.00 & 1.00 & 0.00 & 1.00 \\ 0.84 & 1.34 & 0.35 & 1.09 & 0.21 & 1.48 \\ 0.91 & -0.62 & 1.70 & 0.70 & 0.02 & 1.01 \\ 0.14 & -1.09 & 1.38 & 1.08 & 0.40 & 0.80 \end{bmatrix}$$

The output from the multi-head attention block $\text{Attention}(X_{\text{in}})$ is:

$$\text{Attention}(X_{\text{in}}) = \begin{bmatrix} -0.08 & 0.29 & 0.18 & 0.35 & 0.00 & -0.33 \\ -0.10 & 0.42 & 0.10 & 0.43 & -0.01 & -0.25 \\ -0.03 & 0.31 & 0.08 & 0.29 & 0.07 & -0.10 \\ 0.05 & 0.06 & 0.18 & 0.13 & 0.18 & -0.11 \end{bmatrix}$$

These two matrices are added together element by element, writing the newly discovered contextual relationships directly into the original vector representations:

$$X_{\text{out}} = \begin{bmatrix} 0.02 & 1.29 & 0.18 & 1.35 & 0.00 & 0.67 \\ 0.74 & 1.76 & 0.45 & 1.52 & 0.20 & 1.23 \\ 0.88 & -0.31 & 1.78 & 0.99 & 0.09 & 0.91 \\ 0.19 & -1.03 & 1.56 & 1.21 & 0.58 & 0.69 \end{bmatrix}$$

The Geometric Implications of Addition

Adding the attention output to the original embedding performs vector translation. The attention block calculates a directional shift based on the surrounding context. Adding this shift vector to the original token vector moves the token to a new location in the d_{model} -dimensional space.

The vector for the token `woke` originally represented the static embedding of waking at position `2`. Adding the attention output translates that vector along a direction that incorporates its syntactic relationship with `i` and `up`. The foundational token identity remains intact, and the shifted coordinate reflects its contextual role in the sequence.

Gradient Flow and the Residual Highway

Deep neural networks learn by calculating the gradient of the loss function and propagating that error signal backward through the layers. In a strictly sequential architecture, the gradient multiplies by the derivative of each layer. A sub-layer operation frequently yields a derivative matrix containing small values. An idealized sub-layer Jacobian matrix containing diagonal values of `0.10` illustrates this decay:

$$\text{Jacobian} = \begin{bmatrix} 0.10 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.10 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.10 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.10 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.10 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.10 \end{bmatrix}$$

Backpropagating an error signal through four consecutive layers of this type requires multiplying the Jacobian matrix by itself four times ($0.10^4 = 0.0001 \approx 0.00$). This exponentiation causes the gradient to decay exponentially. The resulting matrix demonstrates signal loss at two-decimal precision, leaving the earliest layers unable to update their parameters:

$$\text{Sequential Gradient Multiplier} = \begin{bmatrix} 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \end{bmatrix}$$

The residual addition neutralizes this vanishing gradient problem. Differentiating an addition operation distributes the upstream gradient equally to both branches. Differentiating the residual equation $X_{\text{out}} = X_{\text{in}} + \text{Attention}(X_{\text{in}})$ with respect to X_{in} assigns an identity derivative I to the skip path. This additive identity term shifts the single-layer gradient multiplier from the sub-layer Jacobian alone to the sum of the identity matrix and the Jacobian:

$$\text{Residual Multiplier} = I + \text{Jacobian}$$

Backpropagating through four layers utilizing residual connections multiplies this updated term by itself four times ($1.10^4 = 1.4641 \approx 1.46$). The addition of the Identity matrix ensures the gradient survives the backward pass:

$$\text{Residual Gradient Multiplier} = \begin{bmatrix} 1.46 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 1.46 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 1.46 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 1.46 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 1.46 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 1.46 \end{bmatrix}$$

The error signal reaches the initial embedding matrices intact. The residual stream functions as an unimpeded gradient highway: contextual information accumulates forward across layers, and loss gradients propagate backward to update early parameters.

This central memory bus ensures that every subsequent layer has direct access to both the initial embeddings and the accumulated contextual updates from all previous layers. Stabilizing these shifted vectors requires layer normalization, which is examined next.

Chapter 9: Taming the Stream: The Geometry of Layer Normalization

The previous section detailed the Residual Stream. The Attention block operates as an independent module that reads from the central memory bus, calculates contextual updates, and adds those updates directly back into the original embeddings. This additive process ensures that the network never loses the raw initial information about the token and its position.

Continuous vector addition introduces a geometric side effect. Repeated updates from attention and feed-forward blocks cause token vector magnitudes to grow across layers, shifting component distributions away from zero. Unbounded or skewed activations saturate downstream linear projections and nonlinearities, triggering numerical instability and exploding or vanishing gradients during backpropagation.

Layer Normalization counteracts this activation drift by standardizing each token vector before the next sub-layer transformation.

The Geometry of Normalization

The token embeddings function as points in a six-dimensional space, where $d_{model} = 6$. Before the addition of the Attention output, these points were bounded by the properties of the initial embedding and positional encoding. Adding the attention output shifts these points away from their initial centered distribution.

The current state of the Residual Stream for the sequence `<BOS>` `i` `woke` `up` is:

$$\text{Residual Stream} = \begin{bmatrix} 0.02 & 1.29 & 0.18 & 1.35 & 0.00 & 0.67 \\ 0.74 & 1.76 & 0.45 & 1.52 & 0.20 & 1.23 \\ 0.88 & -0.31 & 1.78 & 0.99 & 0.09 & 0.91 \\ 0.19 & -1.03 & 1.56 & 1.21 & 0.58 & 0.69 \end{bmatrix}$$

Layer Normalization stabilizes these representations by executing two operations independently on each token vector: subtracting the row mean to center the distribution at zero, and dividing by the row standard deviation to normalize the spread to unit variance.

Layer Normalization operates across the embedding dimension d_{model} for each individual token rather than across the sequence length. Normalizing the vector for `i` is statistically independent of normalizing the vector for `woke`. Per-token normalization preserves strict causal independence across sequence positions between attention layers.

Position	Token	Row Sum	Mean (μ)	Variance (σ²)	Std Dev (σ)
Row 1	`<BOS>`	`3.51`	`0.58` (`0.585`)	`0.32`	`0.57`
Row 2	`i`	`5.90`	`0.98`	`0.32`	`0.56`
Row 3	`woke`	`4.34`	`0.72`	`0.45`	`0.67`
Row 4	`up`	`3.20`	`0.53`	`0.68`	`0.83`

Step 1: Centering the Vector

For a token vector $x \in \mathbb{R}^{d_{\text{model}}}$, the mean μ is the arithmetic average across its $d_{\text{model}} = 6$ components.

The mean for each of the four tokens is:

$$\text{Means} = \begin{bmatrix} 0.58 \\ 0.98 \\ 0.72 \\ 0.53 \end{bmatrix}$$

Subtracting this mean from every element in the corresponding token vector shifts the vector through six-dimensional space so that its components are centered around zero. The relative spacing between the components of the vector remains intact while anchoring the vector to the origin of the coordinate system.

Step 2: Scaling the Vector

Centering resolves the mean drift, yet the dispersion of the vector components may still be too wide or too narrow. Standardizing the scale requires computing the population variance σ^2 of each vector across its d_{model} components.

The variances for the tokens are as follows:

$$\text{Variances} = \begin{bmatrix} 0.32 \\ 0.32 \\ 0.45 \\ 0.68 \end{bmatrix}$$

The vector is scaled by dividing each centered component by the standard deviation $\sqrt{\sigma^2 + \epsilon}$, where a small constant ϵ prevents division by zero:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Evaluating this formula at full precision for each row of the Residual Stream and rounding to two decimal places yields the standardized matrix:

$$\text{Normalized Stream} = \begin{bmatrix} -1.00 & 1.25 & -0.72 & 1.35 & -1.04 & 0.15 \\ -0.43 & 1.38 & -0.95 & 0.95 & -1.39 & 0.44 \\ 0.23 & -1.54 & 1.57 & 0.40 & -0.94 & 0.28 \\ -0.42 & -1.89 & 1.24 & 0.82 & 0.06 & 0.19 \end{bmatrix}$$

Every token row in this standardized matrix now has a mean of 0 and a variance of 1.

Step 3: Learned Scale and Bias

Standardizing the vectors to zero mean and unit variance guarantees numerical stability. Restricting every token vector to unit variance can discard useful representation capacity that the network encodes in the scale or offset of specific features.

Layer Normalization restores this expressive capacity by introducing two learnable parameter vectors of length d_{model}: a scale parameter γ and a bias parameter β:

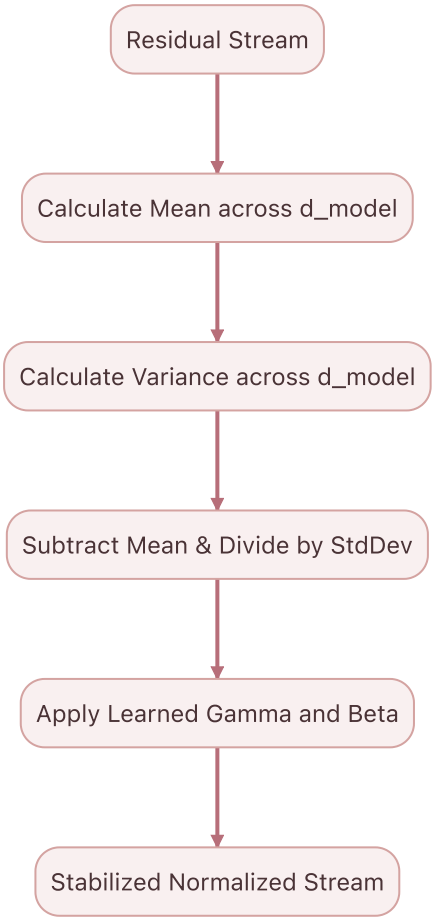
$$\text{Output} = \gamma \odot \hat{x} + \beta$$

Backpropagation updates γ and β during training so the network can amplify or shift individual feature dimensions whenever strict standardization is suboptimal.

In this toy walkthrough, γ is initialized to a vector of ones ($\mathbf{1}$) and β to a vector of zeros ($\mathbf{0}$). This identity initialization leaves the `Normalized Stream` matrix unchanged as the direct input X_{norm} to the Multi-Layer Perceptron.

The Stabilized Backbone

With Layer Normalization complete, the token representations are numerically bounded and ready to enter the next component of the Transformer architecture.



The vectors have been stabilized while retaining the contextual updates harvested by the Attention mechanism. These standardized vectors next enter the Feed-Forward Network, which functions as a conceptual Key-Value memory bank for each individual token.

Chapter 10: The Residual Stream and MLP Expansion

The calculations outlined in Parts 8 and 9 update the residual stream and stabilize the geometry of the network through layer normalization. These operations preserve forward information flow while neutralizing gradient degradation, leaving the token vectors positioned for the next architectural phase.

The MLP as a Key-Value Memory Bank

The model has standardized the residual stream into centered, unit-variance token vectors. Up to this stage, self-attention has routed contextual information between sequence positions: the representation for `up` has pulled in contextual features from `woke`. Self-attention routes information across tokens, yet linear attention mixing alone cannot non-linearly transform those combined features into higher-order semantic concepts.

Processing each contextualized token vector individually requires the second major sub-layer of the Transformer block: the Feed-Forward Network, also called the Multi-Layer Perceptron (MLP).

Introductory treatments often describe the MLP merely as a two-layer feed-forward network that expands dimensionality and applies a non-linear activation. Mechanistic interpretability provides a concrete geometric framing: the MLP functions as a learned Key-Value memory bank stored within its weight matrices. This section examines the first linear projection of the MLP, which serves as the Key lookup stage of that memory retrieval system.

The Geometry of the Keys

The model dimension is $d_{model} = 6$. Standard Transformer architectures expand the hidden layer of the MLP by a factor of four relative to the residual stream, setting the feed-forward dimension to $d_{ff} = 4 \times d_{model} = 24$.

The first projection matrix, W_1, has a shape of 6×24. The calculation multiplies the normalized residual stream X_{norm} (4×6) by W_1, resulting in a projected matrix of shape 4×24.

Rather than viewing W_1 as a monolithic block, examining its column structure clarifies its geometric role. The matrix W_1 consists of 24 distinct column vectors in $\mathbb{R}^6$, where each column represents a learned Key pattern.

Each Key column represents a learned spatial pattern in $\mathbb{R}^6$. Computing the dot product between a normalized token vector and a Key column measures their directional alignment, testing whether the contextualized token exhibits the specific feature combination encoded by that Key.

Defining the first column of the learned W_1 matrix as the key vector k_1 illustrates this lookup mechanism:

$$k_1 = \begin{bmatrix} 0.33 \\ -0.51 \\ -1.89 \\ 0.67 \\ 0.86 \\ 1.55 \end{bmatrix}$$

The system extracts the normalized vector for the third token, `woke`, from Row 3 of the X_{norm} matrix calculated in Chapter 9:

$$x_{woke} = [0.23 \quad -1.54 \quad 1.57 \quad 0.40 \quad -0.94 \quad 0.28]$$

Computing the dot product between x_{woke} and k_1 quantifies how strongly the contextualized `woke` token aligns with the first Key pattern:

$$x_{woke} \cdot k_1 = (0.23 \times 0.33) + (-1.54 \times -0.51) + (1.57 \times -1.89) + (0.40 \times 0.67) + (-0.94 \times 0.86) + (0.28 \times$$

$$x_{woke} \cdot k_1 = 0.08 + 0.79 - 2.97 + 0.27 - 0.81 + 0.43 = -2.21$$

A negative dot product indicates that the vector for `woke` points away from k_1 in six-dimensional space, meaning this token does not match the feature pattern represented by the first Key.

Performing this matrix multiplication across all four rows simultaneously checks every token against all 24 Keys.

The Projection Calculation

The complete matrix multiplication $X_{norm}W_1 = X_{proj}$ follows:

$$X_{proj} = \begin{bmatrix} 0.64 & -0.30 & -0.93 & -0.86 & 0.53 & -1.24 & 0.88 & 0.71 & -2.13 & \dots & -0.19 \\ 1.07 & -1.32 & -0.65 & -0.96 & 1.47 & -2.19 & 2.16 & -0.20 & -2.64 & \dots & 1.73 \\ -2.21 & -0.10 & -2.12 & 0.33 & -2.18 & 2.73 & -0.92 & 1.72 & 3.32 & \dots & 1.09 \\ -0.62 & -0.74 & -0.58 & -0.56 & 0.00 & 2.26 & -0.71 & 0.74 & 2.40 & \dots & 1.53 \end{bmatrix}$$

Each row in X_{proj} corresponds to one of the four sequence tokens, and each column corresponds to one of the 24 Keys. The entry at Row 3, Column 1 equals -2.21, matching the manual dot product computed between `woke` and k_1.

Row 3, Column 9 exhibits a strong positive score of 3.32, indicating that the contextualized representation for `woke` aligns closely with the ninth Key vector.

The Bias Vector

In a standard linear layer, a learned bias vector b_1 is added immediately after the matrix multiplication. The bias vector shifts the pre-activation scores, acting as a baseline activation threshold for each of the 24 Keys:

$$X_{proj_biased} = X_{proj} + b_1$$

A negative bias raises the activation threshold for a Key, requiring a large positive dot product to overcome the offset, whereas a positive bias lowers the threshold so the Key fires more readily.

Adding the learned bias vector b_1 across every row of X_{proj} produces the pre-activation state X_{proj_biased} :

$$b_1 = [-0.06 \quad 0.04 \quad -0.16 \quad -0.04 \quad 0.03 \quad 0.08 \quad 0.14 \quad 0.00 \quad -0.12 \quad \dots \quad 0.15]$$

$$X_{proj_biased} = \begin{bmatrix} 0.58 & -0.26 & -1.09 & -0.90 & 0.56 & -1.16 & 1.03 & 0.71 & -2.25 & \dots & -0.04 \\ 1.01 & -1.28 & -0.81 & -1.00 & 1.50 & -2.11 & 2.30 & -0.21 & -2.75 & \dots & 1.88 \\ -2.27 & -0.07 & -2.28 & 0.29 & -2.16 & 2.81 & -0.78 & 1.72 & 3.20 & \dots & 1.24 \\ -0.69 & -0.71 & -0.74 & -0.60 & 0.03 & 2.34 & -0.56 & 0.74 & 2.29 & \dots & 1.68 \end{bmatrix}$$

The token vectors have now queried all 24 Key patterns in the memory bank. The next stage applies a non-linear activation gate to zero out negative non-matches before projecting the surviving activations back into the residual stream.

Chapter 11: Layer Normalization and The Multi-Layer Perceptron

The calculations established in Parts 9 and 10 center and scale the residual stream while mapping the token representations into a 24-dimensional key space. This geometric expansion isolates conceptual patterns, preparing the network to filter and extract the matched semantic features.

The Multi-Layer Perceptron: Activation and Contraction

Chapter 10 framed the first half of the Multi-Layer Perceptron as a Key-Value memory bank. Projecting the $d_{model} = 6$ residual stream into the $d_{ff} = 24$ intermediate space using W_1 and b_1 evaluated each token against 24 learned Key vectors.

At this stage, token vectors reside in the expanded 24-dimensional space. Two operations complete the MLP block:

1. **Non-Linear Gating:** Apply an element-wise activation threshold to retain positive Key matches and zero out negative non-matches.
2. **Value Contraction:** Multiply the sparse 24-dimensional activations by the Value matrix W_2 (24×6) and add bias b_2 to return a 6-dimensional update vector for the residual stream.

The Non-Linear Gate: ReLU

Linear transformations alone cannot implement selective memory retrieval. Chaining the W_1 projection directly into a second matrix W_2 without an intermediate non-linearity collapses the two matrices into a single linear transformation $W_{\text{eff}} = W_1 W_2$ of rank at most 6 , negating the 24-dimensional expansion. Selective feature activation requires a non-linear gate.

The toy model uses the Rectified Linear Unit (ReLU):

$$\text{ReLU}(x) = \max(0, x)$$

The ReLU function operates as a piecewise-linear gate: negative pre-activation scores indicate absent Key patterns and are clamped to 0 , whereas positive scores pass through unchanged with unit slope.

The biased pre-activation tensor $X_{\text{proj_biased}}$ from Chapter 10 holds the raw Key alignment scores for `<BOS>`, `i`, `woke`, and `up` across all 24 dimensions:

$$X_{\text{proj_biased}} = \begin{bmatrix} 0.58 & -0.26 & -1.09 & -0.90 & 0.56 & -1.16 & 1.03 & 0.71 & -2.25 & \dots & -0.04 \\ 1.01 & -1.28 & -0.81 & -1.00 & 1.50 & -2.11 & 2.30 & -0.21 & -2.75 & \dots & 1.88 \\ -2.27 & -0.07 & -2.28 & 0.29 & -2.16 & 2.81 & -0.78 & 1.72 & 3.20 & \dots & 1.24 \\ -0.69 & -0.71 & -0.74 & -0.60 & 0.03 & 2.34 & -0.56 & 0.74 & 2.29 & \dots & 1.68 \end{bmatrix}$$

Applying the ReLU function element-wise across the tensor yields the sparse activated state X_{act} :

$$X_{act} = \max(0, X_{\text{proj_biased}}) = \begin{bmatrix} 0.58 & 0.00 & 0.00 & 0.00 & 0.56 & 0.00 & 1.03 & 0.71 & 0.00 & \dots & 0.00 \\ 1.01 & 0.00 & 0.00 & 0.00 & 1.50 & 0.00 & 2.30 & 0.00 & 0.00 & \dots & 1.88 \\ 0.00 & 0.00 & 0.00 & 0.29 & 0.00 & 2.81 & 0.00 & 1.72 & 3.20 & \dots & 1.24 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.03 & 2.34 & 0.00 & 0.74 & 2.29 & \dots & 1.68 \end{bmatrix}$$

This operation sparsifies the representation. All negative scores are zeroed out, silencing memory slots that did not match. The remaining positive entries represent specific contextual features recognized by the W_1 Keys.

The Value Matrix: Contracting Back to the Stream

Translating these sparse activations back into the $d_{model} = 6$ residual stream requires the second linear projection matrix W_2 and its bias vector b_2 .

Within the Key-Value memory framework, the columns of W_1 act as the Keys and the rows of W_2 act as the corresponding Values.

The matrix W_2 has dimensions $d_{ff} \times d_{model} = 24 \times 6$, forming a table of 24 six-dimensional Value vectors. Each positive activation scalar in X_{act} scales its corresponding row in W_2 , and summing those weighted rows produces a 6-dimensional update vector for each token.

The following tensors show the first three rows and final row of the 24×6 matrix W_2 alongside the bias vector b_2 (the complete 24-row W_2 matrix is printed in Chapter 13):

$$W_2 = \begin{bmatrix} 0.75 & 0.19 & 0.34 & 0.44 & -0.42 & -0.78 \\ -0.56 & 0.17 & 0.53 & 0.06 & -0.68 & 0.70 \\ -0.80 & -0.24 & -0.28 & 0.13 & 0.55 & -0.75 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ -0.02 & 0.27 & -0.68 & -0.35 & 0.05 & 0.14 \end{bmatrix}$$

$$b_2 = \begin{bmatrix} -0.06 & 0.12 & -0.15 & -0.08 & 0.04 & 0.09 \end{bmatrix}$$

Multiplying the activated memory state X_{act} by the Value matrix W_2 and adding the bias b_2 contracts the representations back down to the $d_{model} = 6$ dimension:

$$X_{contracted} = X_{act} W_2 + b_2$$

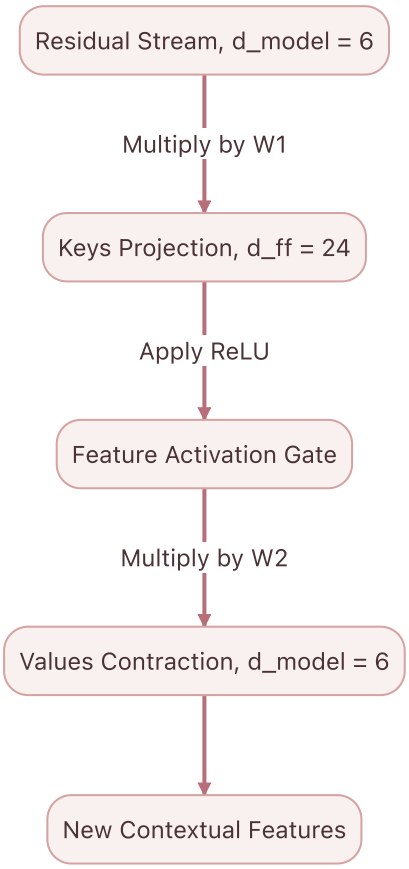
Evaluating the full 24-dimensional matrix multiplication yields the Multi-Layer Perceptron output tensor:

$$X_{contracted} = \begin{bmatrix} -4.07 & 0.04 & -0.06 & -1.74 & -1.17 & -0.77 \\ -6.51 & 0.18 & -0.33 & -3.85 & -0.84 & -1.43 \\ 0.82 & -3.22 & 0.39 & -2.11 & 3.22 & 6.02 \\ 1.18 & -2.83 & -0.72 & -1.89 & 1.58 & 4.39 \end{bmatrix}$$

This 4×6 matrix holds the contracted feature updates for all four sequence positions. Row 3, corresponding to the token `woke`, now stores the weighted combination of Value vectors triggered by its active Key matches.

The Big Picture of the Multi-Layer Perceptron

The complete Key-Value workflow operates as a three-stage expansion, gating, and contraction pipeline:



The Multi-Layer Perceptron reads from the normalized residual stream, expands the representation to probe 24 high-dimensional Keys, filters those matches through a non-linear gate, and contracts the surviving Values back into a 6-dimensional update vector.

The next step writes this update matrix back into the central residual highway and normalizes the result, completing Layer 1.

Chapter 12: The MLP as a Key-Value Memory and Layer 1 Completion

Complete Key-Value Memory Synthesis

Self-attention enables tokens to gather context across the sequence and shift their geometric positions based on surrounding words. Layer Normalization then standardizes those context-enriched vectors before they enter the Multi-Layer Perceptron (MLP). Viewed through mechanistic interpretability, the two linear layers of the MLP form a complete Key-Value memory bank: the expansion matrix W_1 (6×24) stores 24 six-dimensional Key columns, the ReLU gate filters out negative alignments, and the contraction matrix W_2 (24×6) writes the matched Value rows back into the $d_{model} = 6$ subspace.

Inspecting the 10-column view of the calibrated W_1 matrix alongside the resulting pre-activation state $X_{\text{proj_biased}}$ consolidates the forward trace of the memory lookup stage before the output merges back into the residual stream:

$$W_1 = \begin{bmatrix} 0.33 & -0.71 & 0.20 & 0.15 & 0.71 & -0.90 & 0.55 & -0.72 & -0.44 & \dots & 1.34 \\ -0.51 & 1.01 & -0.44 & 0.44 & -1.17 & 0.04 & -0.53 & 0.52 & -0.34 & \dots & -1.68 \\ -1.89 & 1.56 & -1.29 & 0.83 & -2.77 & 2.09 & -1.75 & 1.56 & 1.94 & \dots & -2.10 \\ 0.67 & -0.70 & -0.02 & -0.55 & 1.07 & -0.56 & 0.27 & -0.11 & -0.79 & \dots & 0.85 \\ 0.86 & -0.17 & 1.20 & -0.25 & 1.14 & -0.19 & -0.09 & -0.87 & -0.43 & \dots & 0.06 \\ 1.55 & -2.53 & 1.14 & -1.22 & 3.02 & -0.86 & 2.47 & -1.93 & -0.83 & \dots & 4.32 \end{bmatrix}$$

Multiplying the normalized sequence representation X_{norm} for `<BOS>`, `i`, `woke`, and `up` by W_1 and adding the bias vector b_1 produces the 24 pre-activation potentials in $X_{\text{proj_biased}}$:

$$\text{Projected State } (X_{\text{proj_biased}}) = \begin{bmatrix} 0.58 & -0.26 & -1.09 & -0.90 & 0.56 & -1.16 & 1.03 & 0.71 & -2.25 \\ 1.01 & -1.28 & -0.81 & -1.00 & 1.50 & -2.11 & 2.30 & -0.21 & -2.75 \\ -2.27 & -0.07 & -2.28 & 0.29 & -2.16 & 2.81 & -0.78 & 1.72 & 3.20 \\ -0.69 & -0.71 & -0.74 & -0.60 & 0.03 & 2.34 & -0.56 & 0.74 & 2.29 \end{bmatrix}$$

Positive entries in this matrix mark strong directional matches between a token and the corresponding Key column, whereas negative entries mark absent features that the ReLU activation clamps to zero. Projecting the surviving positive activations through the Value matrix W_2 and adding b_2 (as computed in Chapter 11) yields the 4×6 update matrix MLP_{output} .

The Information Accumulator

The Transformer does not pass data sequentially through destructive filters that overwrite prior representations. It maintains a persistent residual vector for each token, and each sub-layer reads from this vector and adds its computed update back into the stream.

The output of the MLP is an additive update rather than a replacement for the representation of the token. The architecture adds the MLP output matrix directly to the Residual Stream state X_1 that exited the first attention residual connection in Chapter 8.

The residual stream entering this addition is X_1 , which holds the positionally encoded embeddings enriched with the Layer 1 self-attention updates:

$$X_1 = \begin{bmatrix} 0.02 & 1.29 & 0.18 & 1.35 & 0.00 & 0.67 \\ 0.74 & 1.76 & 0.45 & 1.52 & 0.20 & 1.23 \\ 0.88 & -0.31 & 1.78 & 0.99 & 0.09 & 0.91 \\ 0.19 & -1.03 & 1.56 & 1.21 & 0.58 & 0.69 \end{bmatrix}$$

The MLP contracted output matrix from Chapter 11 provides the feature updates discovered within the local context of each token:

$$MLP_{output} = \begin{bmatrix} -4.07 & 0.04 & -0.06 & -1.74 & -1.17 & -0.77 \\ -6.51 & 0.18 & -0.33 & -3.85 & -0.84 & -1.43 \\ 0.82 & -3.22 & 0.39 & -2.11 & 3.22 & 6.02 \\ 1.18 & -2.83 & -0.72 & -1.89 & 1.58 & 4.39 \end{bmatrix}$$

Adding X_1 and MLP_{output} element by element produces the updated Residual Stream X_2 :

$$X_2 = X_1 + MLP_{output} = \begin{bmatrix} -4.05 & 1.33 & 0.12 & -0.39 & -1.17 & -0.10 \\ -5.77 & 1.94 & 0.12 & -2.33 & -0.64 & -0.20 \\ 1.70 & -3.53 & 2.17 & -1.12 & 3.31 & 6.93 \\ 1.37 & -3.86 & 0.84 & -0.68 & 2.16 & 5.08 \end{bmatrix}$$

The magnitudes in the bottom two rows, representing the tokens `woke` and `up`, have grown substantially. The MLP has injected a strong semantic update into these token representations based on their local context.

Preparing for Layer 2 Normalization

Accumulating updates through vector addition increases activation magnitudes across sub-layers. Passing unnormalized vectors with variances as large as 10.89 directly into Layer 2 would inflate the Query-Key dot products in the next attention block, triggering the Softmax saturation problem analyzed in Chapter 4.

Applying a second Layer Normalization operation before Layer 2 restores zero mean and unit variance across the $d_{model} = 6$ dimension for each token independently:

$$\text{Means} = \begin{bmatrix} -0.71 \\ -1.15 \\ 1.57 \\ 0.82 \end{bmatrix} \quad \text{Variances} = \begin{bmatrix} 2.78 \\ 5.85 \\ 10.89 \\ 7.40 \end{bmatrix}$$

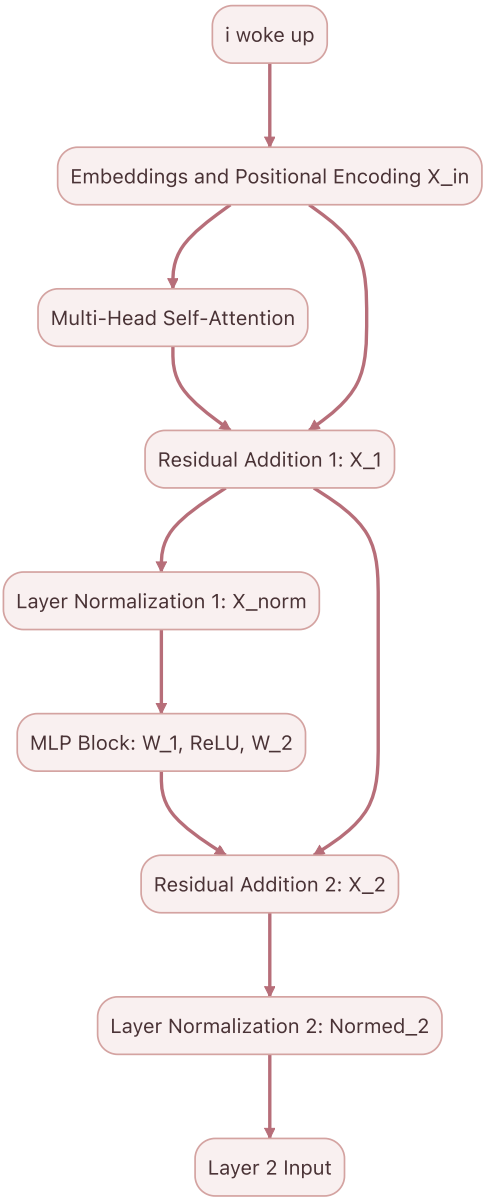
Subtracting the row mean and dividing by the row standard deviation recenters each token vector at zero and scales its spread to unit variance:

$$Normed_2 = \begin{bmatrix} -2.00 & 1.22 & 0.50 & 0.19 & -0.28 & 0.37 \\ -1.91 & 1.28 & 0.52 & -0.49 & 0.21 & 0.39 \\ 0.04 & -1.55 & 0.18 & -0.82 & 0.52 & 1.62 \\ 0.20 & -1.72 & 0.01 & -0.55 & 0.49 & 1.57 \end{bmatrix}$$

The directional relationships encoded across the six coordinates are preserved, while the overall vector magnitudes return to a well-conditioned numerical range.

Visualizing the Complete Layer 1 Architecture

The sequence of operations in the first layer of the Transformer is now complete. The full Layer 1 computation graph traces how tensors flow from the initial prompt to the normalized input of Layer 2:



The vectors exiting Layer 1 are no longer static dictionary lookups; they are contextualized geometric representations. The vector for the token `woke` now encodes relational features from the preceding pronoun `i` alongside local conceptual updates from the MLP. Passing this normalized 4×6 matrix into Layer 2 allows the next self-attention block to compute higher-order associations across multi-token phrases.

Chapter 13: The MLP Activation and Layer 2 Self-Attention

The projection matrix from Chapter 10 expanded the normalized residual stream into a twenty-four-dimensional space, querying a learned bank of conceptual keys. The resulting biased projection matrix contained a spectrum of positive and negative values representing the alignment strength with each learned pattern. The network must now determine which of these candidate patterns apply to the current sequence context and discard the rest. This non-linear filtering operation is executed by the activation function.

Non-Linear Filtering and Value Contraction

The toy model employs the Rectified Linear Unit (ReLU), defined as $\mathbf{ReLU}(z) = \mathbf{max}(0, z)$. Every negative entry is clamped to zero, whereas positive entries pass through unchanged. Mechanistically, ReLU enforces a hard firing threshold. A token vector that fails to align with a specific conceptual key produces a negative dot product, which the ReLU threshold clamps to zero. That neuron remains inactive.

Applying this element-wise non-linearity to the biased projection state produces the sparse activated memory representation:

$$\mathbf{Activated\ State} = \begin{bmatrix} 0.58 & 0.00 & 0.00 & 0.00 & 0.56 & 0.00 & 1.03 & 0.71 & 0.00 & \dots & 0.00 \\ 1.01 & 0.00 & 0.00 & 0.00 & 1.50 & 0.00 & 2.30 & 0.00 & 0.00 & \dots & 1.88 \\ 0.00 & 0.00 & 0.00 & 0.29 & 0.00 & 2.81 & 0.00 & 1.72 & 3.20 & \dots & 1.24 \\ 0.00 & 0.00 & 0.00 & 0.00 & 0.03 & 2.34 & 0.00 & 0.74 & 2.29 & \dots & 1.68 \end{bmatrix}$$

The hidden activation landscape is now sparse: only positive conceptual matches survive. Zeroed entries correspond to features irrelevant to a token in its current context, preventing contradictory signals from propagating into the value projection.

With the active keys selected, the network enters the extraction phase. The second half of the Multi-Layer Perceptron (MLP) is parameterized by the 24×6 weight matrix W_2 , which projects the twenty-four-dimensional hidden space back down to the model dimension $d_{\text{model}} = 6$. In the Key-Value memory analogy, the columns of W_1 act as the Keys that detect patterns, and the rows of W_2 store the corresponding Values written back into the residual stream:

$$W_2 = \begin{bmatrix} 0.75 & 0.19 & 0.34 & 0.44 & -0.42 & -0.78 \\ -0.56 & 0.17 & 0.53 & 0.06 & -0.68 & 0.70 \\ -0.80 & -0.24 & -0.28 & 0.13 & 0.55 & -0.75 \\ -0.16 & 0.16 & -0.58 & -0.05 & 0.09 & -0.67 \\ -0.65 & -0.03 & -0.11 & 0.11 & 1.11 & 0.45 \\ 0.22 & -0.36 & 0.47 & -0.39 & 0.78 & 0.59 \\ -1.00 & -0.59 & -0.10 & -0.20 & -0.63 & -0.61 \\ 0.36 & -0.27 & -0.76 & -0.14 & 0.48 & 0.84 \\ 0.03 & -0.16 & -0.40 & -0.26 & 0.27 & 0.63 \\ 0.62 & -0.12 & 0.44 & 0.38 & 0.23 & -1.23 \\ -0.03 & -0.14 & 0.66 & -0.14 & 0.23 & 0.13 \\ 0.63 & 0.38 & 0.51 & 0.75 & 0.48 & 0.39 \\ -0.13 & 0.05 & 1.08 & 0.02 & 0.25 & -0.04 \\ -0.15 & -1.50 & 0.86 & -0.59 & -0.42 & 0.68 \\ -1.18 & 0.01 & -0.28 & -0.92 & 0.31 & -0.39 \\ -0.05 & -0.21 & 0.32 & 1.13 & -0.63 & 0.43 \\ -0.35 & -0.18 & -0.95 & -0.57 & -0.22 & 0.41 \\ 0.12 & 0.29 & 0.37 & -0.69 & 0.73 & -0.16 \\ 0.34 & -0.09 & 0.13 & 0.32 & -1.13 & 1.01 \\ 0.09 & 0.55 & 0.11 & 0.04 & -0.66 & -0.20 \\ -1.11 & -0.08 & 0.79 & -0.94 & -0.34 & -0.19 \\ 0.58 & 0.62 & 0.03 & 0.05 & -0.08 & -0.24 \\ -0.24 & -0.23 & -0.08 & 0.04 & -0.32 & 0.31 \\ -0.02 & 0.27 & -0.68 & -0.35 & 0.05 & 0.14 \end{bmatrix}$$

Each active neuron in the hidden layer retrieves its corresponding row from the full 24-row Value matrix W_2 introduced in Chapter 11. Multiplying the activated state by W_2 and adding the bias vector b_2 (`[-0.06, 0.12, -0.15, -0.08, 0.04, 0.09]`) computes a weighted sum of Value rows, producing the consolidated 4×6 update matrix for the residual stream:

$$\text{Contracted Output} = \begin{bmatrix} -4.07 & 0.04 & -0.06 & -1.74 & -1.17 & -0.77 \\ -6.51 & 0.18 & -0.33 & -3.85 & -0.84 & -1.43 \\ 0.82 & -3.22 & 0.39 & -2.11 & 3.22 & 6.02 \\ 1.18 & -2.83 & -0.72 & -1.89 & 1.58 & 4.39 \end{bmatrix}$$

Gated Feed-Forward Networks: SwiGLU

Foundational Transformer architectures apply a single ReLU threshold, whereas contemporary large language models frequently substitute gated activation units such as SwiGLU (Swish-Gated Linear Unit). SwiGLU replaces the single expansion matrix W_1 with two parallel linear projections:

- **Feature Projection (xW_{up}):** Maps the normalized token vector into the expanded intermediate space to propose candidate conceptual features.
- **Gate Projection ($\text{Swish}(xW_{\text{gate}})$):** Maps the same input through an independent weight matrix and applies the smooth non-linear $\text{Swish}(z) = z \cdot \sigma(z)$ activation to compute a continuous gating scalar for each hidden dimension.

Element-wise multiplication combines the two branches before the final W_2 contraction projects the gated state back to d_{model} :

$$\text{SwiGLU}(x) = (\text{Swish}(xW_{\text{gate}}) \odot (xW_{\text{up}}))W_2$$

Rather than enforcing a binary zero floor like ReLU, SwiGLU acts as a continuous, context-dependent dial. The feature branch proposes a semantic candidate, and the gate branch independently scales or suppresses that candidate based on token interactions. Decoupling feature generation from feature gating increases the expressive capacity of the feed-forward memory bank.

Whether an architecture uses a ReLU threshold or a SwiGLU gate, the structural role of the MLP remains identical: the block evaluates context-enriched token vectors, retrieves learned conceptual features, and writes those updates into the residual stream.

Transition to Layer 2

In the first layer of the Transformer, the self-attention mechanism evaluated relationships between positionally encoded word embeddings. Projecting the tokens `woke` and `up` into their Layer 1 Query and Key spaces measured their initial lexical affinity. The network then routed those localized updates into the residual stream, refined them through the Key-Value Multi-Layer Perceptron, and standardized their geometry with Layer Normalization. Entering the second layer of self-attention, the token vectors no longer represent isolated dictionary definitions; each row now encodes a contextualized summary of its surrounding linguistic environment.

The Contextualized Input

The input to Layer 2, denoted as X_2 , is the normalized output of the first layer derived in Chapter 12. The vectors occupying this matrix differ substantially from the initial token embeddings. The first row corresponds to `<BOS>`, the second to `i`, the third to `woke`, and the fourth to `up`, and their coordinates now encode the structural and semantic relationships assembled during Layer 1:

$$X_2 = \begin{bmatrix} -2.00 & 1.22 & 0.50 & 0.19 & -0.28 & 0.37 \\ -1.91 & 1.28 & 0.52 & -0.49 & 0.21 & 0.39 \\ 0.04 & -1.55 & 0.18 & -0.82 & 0.52 & 1.62 \\ 0.20 & -1.72 & 0.01 & -0.55 & 0.49 & 1.57 \end{bmatrix}$$

Layer 2 self-attention does not merely test whether `woke` relates to `up` in isolation. The mechanism evaluates how a verb phrase already linked to the subject `i` aligns with the particle `up` to form a compound phrasal concept. Self-attention in deeper layers operates on multi-token abstractions.

The Second Layer Projections

Just as in Layer 1, the network projects these 6-element residual vectors into 2-element head subspaces ($d_k = 2$) to compute attention. A dedicated set of weight matrices initializes the first head of Layer 2. These matrices, $W_Q^{(2)}$, $W_K^{(2)}$, and $W_V^{(2)}$, serve the same geometric role as their Layer 1 counterparts, defining a bilinear form that aligns contextualized semantic vectors in a shared subspace:

$$W_Q^{(2)} = \begin{bmatrix} 0.10 & -0.20 \\ -0.30 & 0.40 \\ 0.50 & -0.10 \\ -0.20 & 0.30 \\ 0.40 & 0.20 \\ -0.10 & -0.50 \end{bmatrix}$$

$$W_K^{(2)} = \begin{bmatrix} -0.20 & 0.30 \\ 0.40 & -0.10 \\ -0.30 & 0.50 \\ 0.10 & -0.40 \\ 0.20 & 0.20 \\ -0.50 & 0.10 \end{bmatrix}$$

$$W_V^{(2)} = \begin{bmatrix} 0.30 & -0.10 \\ -0.20 & 0.40 \\ 0.10 & -0.30 \\ -0.40 & 0.20 \\ 0.50 & -0.20 \\ -0.10 & 0.50 \end{bmatrix}$$

Multiplying the full-precision normalized Layer 1 output X_2 by each weight matrix and rounding to two decimal places produces the Layer 2 Queries Q_2 , Keys K_2 , and Values V_2 .

The Query Space

The Q_2 matrix represents the contextual patterns each token searches for across the sequence:

$$Q_2 = X_2 W_Q^{(2)} = \begin{bmatrix} -0.50 & 0.66 \\ -0.17 & 0.54 \\ 0.77 & -1.60 \\ 0.69 & -1.58 \end{bmatrix}$$

The Key Space

The K_2 matrix represents the compound features each contextualized token advertises to the sequence:

$$K_2 = X_2 W_K^{(2)} = \begin{bmatrix} 0.52 & -0.57 \\ 0.53 & -0.16 \\ -1.47 & 0.85 \\ -1.47 & 0.71 \end{bmatrix}$$

The Value Space

The V_2 matrix stores the contextualized payload each token contributes to downstream positions that attend to it:

$$V_2 = X_2 W_V^{(2)} = \begin{bmatrix} -1.05 & 0.82 \\ -0.51 & 0.60 \\ 0.77 & -0.13 \\ 0.71 & -0.13 \end{bmatrix}$$

A Shift in Abstraction

The linear algebra remains identical to Layer 1: an input tensor projects through three 6×2 transformations to prepare for scaled dot-product attention. The shift lies entirely in the information carried by X_2 . Queries and Keys in Layer 2 no longer match isolated lexical traits; they match multi-token syntactic structures and composite semantic roles. Chapter 14 verifies the Layer 1 to Layer 2 residual boundary and computes the scaled attention scores from Q_2 and K_2 .

Chapter 14: Final Layer 1 Residuals, Norms, and Layer 2 Attention

Auditing the Layer 1 to Layer 2 Boundary

Completing the Multi-Layer Perceptron contraction in Chapter 13 concluded the feature extraction phase of Layer 1. Before scoring the Layer 2 Query and Key projections derived at the end of Chapter 13, this section audits the residual addition and Layer Normalization boundary that produced the Layer 2 input matrix X_2.

Integrating the extracted MLP features with the post-attention representation occurs through the second residual connection of Layer 1. Rather than overwriting the intermediate state, the network adds the contracted MLP output directly to the normalized vectors that entered the feed-forward block. This additive skip connection preserves the contextual identity of each token alongside its newly retrieved conceptual features and maintains a direct gradient highway during backpropagation.

Prior to the feed-forward network, the post-attention residual stream held the following 4×6 matrix (from Chapter 8):

$$\text{Original Residual Stream} = \begin{bmatrix} 0.02 & 1.29 & 0.18 & 1.35 & 0.00 & 0.67 \\ 0.74 & 1.76 & 0.45 & 1.52 & 0.20 & 1.23 \\ 0.88 & -0.31 & 1.78 & 0.99 & 0.09 & 0.91 \\ 0.19 & -1.03 & 1.56 & 1.21 & 0.58 & 0.69 \end{bmatrix}$$

The network adds the contracted 4×6 output from the Multi-Layer Perceptron directly to this stream:

$$\text{MLP Output} = \begin{bmatrix} -4.07 & 0.04 & -0.06 & -1.74 & -1.17 & -0.77 \\ -6.51 & 0.18 & -0.33 & -3.85 & -0.84 & -1.43 \\ 0.82 & -3.22 & 0.39 & -2.11 & 3.22 & 6.02 \\ 1.18 & -2.83 & -0.72 & -1.89 & 1.58 & 4.39 \end{bmatrix}$$

Summing the two matrices element by element produces an enriched composite representation that merges localized conceptual features with the contextual routing established by Layer 1 self-attention:

$$\text{New Residual Stream} = \begin{bmatrix} -4.05 & 1.33 & 0.12 & -0.39 & -1.17 & -0.10 \\ -5.77 & 1.94 & 0.12 & -2.33 & -0.64 & -0.20 \\ 1.70 & -3.53 & 2.17 & -1.12 & 3.31 & 6.93 \\ 1.37 & -3.86 & 0.84 & -0.68 & 2.16 & 5.08 \end{bmatrix}$$

Before these vectors advance into Layer 2, their coordinate distributions must be standardized. Element-wise residual addition shifts the row means and expands the variance of the token representations; left unchecked across stacked layers, this numerical drift destabilizes both forward activations and backward gradients.

Applying Layer Normalization corrects this drift by recalculating the mean and variance independently across the six features of each row and rescaling every row to zero mean and unit variance. The normalization constrains vector magnitudes while preserving the directional geometry encoded in the relative coordinate ratios:

$$\text{Normalized Layer 2 Input } (X_2) = \begin{bmatrix} -2.00 & 1.22 & 0.50 & 0.19 & -0.28 & 0.37 \\ -1.91 & 1.28 & 0.52 & -0.49 & 0.21 & 0.39 \\ 0.04 & -1.55 & 0.18 & -0.82 & 0.52 & 1.62 \\ 0.20 & -1.72 & 0.01 & -0.55 & 0.49 & 1.57 \end{bmatrix}$$

This standardized 4×6 matrix X_2 completes Layer 1 of the Transformer architecture. Each row now integrates lexical identity, sinusoidal position, multi-head contextual routing, and feed-forward memory retrieval.

Passing X_2 into Layer 2 repeats the self-attention and feed-forward sub-layers at a higher level of abstraction. Rather than scoring isolated vocabulary embeddings, Layer 2 scores relationships between composite phrase representations. The remainder of this chapter traces how the Layer 2 Query and Key projections are aligned and scaled.

The Semantic Shift in Queries and Keys

Layer 1 attention derived its Queries Q and Keys K from static token embeddings combined with positional encodings, probing basic pairwise relationships such as matching a subject with a verb. Entering Layer 2, the input vectors have already passed through the first attention block and the Multi-Layer Perceptron, absorbing surrounding sequence context.

Processing the prompt `<BOS>` `i` `woke` `up` to predict the next token now operates on mixed representations: the vectors for `woke` and `up` already share features in the residual stream. Projecting these composite representations into Q_2 and K_2 tests compound syntactic relationships across the phrase.

The Q_2 matrix and the transposed Key matrix K_2^T for the first attention head in Layer 2 (derived in Chapter 13) take the following values:

$$Q_2 = \begin{bmatrix} -0.50 & 0.66 \\ -0.17 & 0.54 \\ 0.77 & -1.60 \\ 0.69 & -1.58 \end{bmatrix}$$

$$K_2^T = \begin{bmatrix} 0.52 & 0.53 & -1.47 & -1.47 \\ -0.57 & -0.16 & 0.85 & 0.71 \end{bmatrix}$$

Calculating the Unscaled Alignment

The mechanism for measuring pairwise relevance remains the inner product. Multiplying Q_2 (4×2) by K_2^T (2×4) computes the raw alignment score between every contextualized query and every contextualized key. Vectors pointing in similar directions within the 2-dimensional head subspace produce positive dot products, opposing vectors produce negative dot products, and orthogonal vectors evaluate to zero.

Evaluating the matrix multiplication $Q_2 \cdot K_2^T$ generates the 4×4 unscaled attention score matrix:

$$\text{Scores}_{\text{unscaled}} = \begin{bmatrix} -0.64 & -0.38 & 1.30 & 1.21 \\ -0.40 & -0.18 & 0.71 & 0.63 \\ 1.31 & 0.67 & -2.48 & -2.27 \\ 1.26 & 0.62 & -2.35 & -2.14 \end{bmatrix}$$

The lower half of this matrix exhibits distinct structural contrast. The rows corresponding to `woke` (Row 3) and `up` (Row 4) assign positive alignment scores (`1.31, 0.67` and `1.26, 0.62`) to the initial anchor positions `<BOS>` and `i` while assigning negative scores (`-2.48, -2.27` and `-2.35, -2.14`) to the verb-particle cluster itself.

Stabilizing the Variance

Before normalizing these scores into probabilities, the scaling factor must be applied. Chapter 4 established that the variance of a dot product grows proportionally with the dimensionality d_k of the projected vectors. Unchecked variance produces large-magnitude dot products in the unscaled score matrix. Passing extreme positive or negative values into the Softmax function drives the output distribution toward a one-hot step function, operating in flat asymptotic regions where the Softmax gradient approaches zero. This Softmax saturation stalls weight updates during backpropagation.

Dividing the unscaled scores by the square root of the head dimension ($\sqrt{d_k} = \sqrt{2} \approx 1.4142$) restores unit variance:

$$\text{Scores}_{\text{scaled}} = \begin{bmatrix} -0.45 & -0.27 & 0.92 & 0.86 \\ -0.28 & -0.13 & 0.50 & 0.45 \\ 0.93 & 0.47 & -1.76 & -1.60 \\ 0.89 & 0.44 & -1.66 & -1.51 \end{bmatrix}$$

Scaling by $\sqrt{d_k}$ preserves the relative alignment rankings while compressing the absolute magnitudes. This variance stabilization keeps the subsequent Softmax distribution in a smooth regime with non-zero gradients, routing information proportionally across multiple tokens rather than collapsing onto a single position.

With the scaled Layer 2 alignment scores established, Chapter 15 applies the lower-triangular causal mask, normalizes the masked scores via Softmax, and multiplies by V_2 to extract the Head 1 output.

Chapter 15: Layer 2 Self-Attention

Advancing into the second layer of the Transformer architecture shifts how the network routes sequence information. Layer 1 operated on static token embeddings combined with sinusoidal positional encodings. The vectors entering Layer 2 are contextualized amalgamations of lexical identity, local syntax, and conceptual features extracted by the Layer 1 Multi-Layer Perceptron.

This evolution of the input representations supports hierarchical abstraction. Layer 1 self-attention resolved immediate lexical pairings across the prompt, whereas Layer 2 self-attention computes similarities between composite phrase structures. Rather than testing how an isolated verb relates to a pronoun, Layer 2 evaluates how a contextualized action cluster aligns with the sequence anchors.

Subspace Tensors Entering the Masking Stage

To keep the complete Layer 2 Head 1 forward trace self-contained for causal masking and Value extraction, this section consolidates the normalized Layer 2 input X_2 and the three subspace projections (Q_2, K_2, V_2) derived in Chapter 13.

The normalized output from Layer 1 serves as the 4×6 input matrix for Layer 2:

$$\text{Layer 2 Input } (X_2) = \begin{bmatrix} -2.00 & 1.22 & 0.50 & 0.19 & -0.28 & 0.37 \\ -1.91 & 1.28 & 0.52 & -0.49 & 0.21 & 0.39 \\ 0.04 & -1.55 & 0.18 & -0.82 & 0.52 & 1.62 \\ 0.20 & -1.72 & 0.01 & -0.55 & 0.49 & 1.57 \end{bmatrix}$$

The self-attention mechanics match Layer 1: three head-specific weight matrices project X_2 into 2-dimensional Query, Key, and Value subspaces.

The Query weight matrix $W_Q^{(2)}$ maps the enriched token vectors into a target-seeking subspace:

$$\text{Layer 2 Head 1 Query Weights } (W_Q^{(2)}) = \begin{bmatrix} 0.10 & -0.20 \\ -0.30 & 0.40 \\ 0.50 & -0.10 \\ -0.20 & 0.30 \\ 0.40 & 0.20 \\ -0.10 & -0.50 \end{bmatrix}$$

Multiplying X_2 by $W_Q^{(2)}$ produces the 4×2 Query vectors for Head 1:

$$\text{Query Vectors } (Q_2) = \begin{bmatrix} -0.50 & 0.66 \\ -0.17 & 0.54 \\ 0.77 & -1.60 \\ 0.69 & -1.58 \end{bmatrix}$$

The Key weight matrix $W_K^{(2)}$ projects X_2 into a complementary indexing subspace where each token broadcasts its composite syntactic role:

$$\text{Layer 2 Head 1 Key Weights } (W_K^{(2)}) = \begin{bmatrix} -0.20 & 0.30 \\ 0.40 & -0.10 \\ -0.30 & 0.50 \\ 0.10 & -0.40 \\ 0.20 & 0.20 \\ -0.50 & 0.10 \end{bmatrix}$$

The resulting 4×2 Key vectors establish the indexing targets matched against Q_2 in Chapter 14:

$$\text{Key Vectors } (K_2) = \begin{bmatrix} 0.52 & -0.57 \\ 0.53 & -0.16 \\ -1.47 & 0.85 \\ -1.47 & 0.71 \end{bmatrix}$$

The Value weight matrix $W_V^{(2)}$ isolates the contextual payload that will be blended once the attention probabilities are finalized:

$$\text{Layer 2 Head 1 Value Weights } (W_V^{(2)}) = \begin{bmatrix} 0.30 & -0.10 \\ -0.20 & 0.40 \\ 0.10 & -0.30 \\ -0.40 & 0.20 \\ 0.50 & -0.20 \\ -0.10 & 0.50 \end{bmatrix}$$

Multiplying X_2 by $W_V^{(2)}$ generates the 4×2 Value vectors for Head 1:

$$\text{Value Vectors } (V_2) = \begin{bmatrix} -1.05 & 0.82 \\ -0.51 & 0.60 \\ 0.77 & -0.13 \\ 0.71 & -0.13 \end{bmatrix}$$

In Chapter 14, taking the inner product $Q_2 K_2^T$ and dividing by $\sqrt{d_k} = \sqrt{2}$ produced the 4×4 scaled attention score matrix. The model now completes Layer 2 Head 1 attention by applying the causal mask, normalizing each row via Softmax, and multiplying by V_2 .

The Causal Mask

Autoregressive training predicts the next token at all sequence positions in parallel, requiring the attention graph to respect the arrow of time. Allowing a token position to attend to future tokens leaks target information backward and invalidates autoregressive training.

Applying a lower-triangular causal mask to the scaled attention scores prevents this look-ahead leakage by setting all entries above the main diagonal to negative infinity ($-\infty$). Exponentiating $-\infty$ during the subsequent Softmax step drives every masked future weight to zero ($e^{-\infty} = 0$).

The scaled attention scores computed in Chapter 14 are:

$$\text{Scores} = \begin{bmatrix} -0.45 & -0.27 & 0.92 & 0.86 \\ -0.28 & -0.13 & 0.50 & 0.45 \\ 0.93 & 0.47 & -1.76 & -1.60 \\ 0.89 & 0.44 & -1.66 & -1.51 \end{bmatrix}$$

Applying the lower-triangular causal mask yields the causally constrained score matrix:

$$\text{Masked} = \begin{bmatrix} -0.45 & -\infty & -\infty & -\infty \\ -0.28 & -0.13 & -\infty & -\infty \\ 0.93 & 0.47 & -1.76 & -\infty \\ 0.89 & 0.44 & -1.66 & -1.51 \end{bmatrix}$$

Normalizing with Softmax

The unmasked entries in each row remain unbounded real numbers. Converting these unbounded real numbers into normalized attention weights requires mapping each row to a probability distribution that sums to 1.0 . The Softmax function exponentiates each score in a row and divides by the row sum of exponentiated scores.

This non-linear normalization amplifies higher alignment scores while suppressing negative scores. Applying Softmax row by row to the masked matrix produces the Layer 2 Head 1 Attention Probabilities:

$$\text{Attention Probabilities} = \begin{bmatrix} 1.00 & 0.00 & 0.00 & 0.00 \\ 0.46 & 0.54 & 0.00 & 0.00 \\ 0.59 & 0.37 & 0.04 & 0.00 \\ 0.55 & 0.35 & 0.04 & 0.05 \end{bmatrix}$$

Inspecting these probability rows highlights two structural patterns (note that Row 4 sums to 0.99 at two-decimal display precision, corresponding to $0.5536 + 0.3530 + 0.0432 + 0.0502 = 1.0000$ at four decimal places). In Row 1, the causal mask restricts `<BOS>` to attend 100% to itself. In Row 4, the final token `up` concentrates its attention near the start of the sequence, allocating 55% of its weight to `<BOS>` and 35% to `i`. Deeper Transformer layers frequently route attention back toward initial boundary and subject tokens to anchor phrase-level representations.

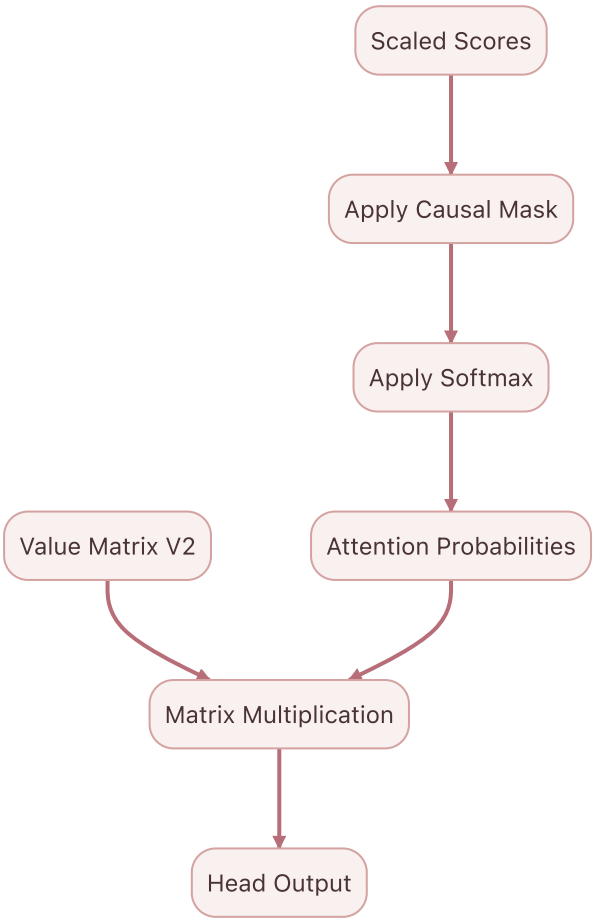
Extracting the Value

The attention probability matrix specifies how much weight each token assigns to preceding positions, and the Value matrix V_2 defines the feature vectors retrieved from those positions.

Multiplying the 4×4 Attention Probabilities matrix by the 4×2 Value matrix V_2 computes a convex combination of Value rows for each sequence position:

$$\text{Head Output} = \begin{bmatrix} 1.00 & 0.00 & 0.00 & 0.00 \\ 0.46 & 0.54 & 0.00 & 0.00 \\ 0.59 & 0.37 & 0.04 & 0.00 \\ 0.55 & 0.35 & 0.04 & 0.05 \end{bmatrix} \begin{bmatrix} -1.05 & 0.82 \\ -0.51 & 0.60 \\ 0.77 & -0.13 \\ 0.71 & -0.13 \end{bmatrix} = \begin{bmatrix} -1.05 & 0.82 \\ -0.76 & 0.70 \\ -0.78 & 0.70 \\ -0.69 & 0.65 \end{bmatrix}$$

This dataflow executes as a four-stage sequence of matrix operations:



This 4×2 output matrix encodes a multi-token feature mixture for Head 1. Row 4, corresponding to `up`, combines the contextual Value representations of `<BOS>`, `i`, `woke`, and `up` after two rounds of self-attention and the Layer 1 Multi-Layer Perceptron.

In Chapter 16, the projected Layer 2 attention updates merge into the residual stream and pass through the Layer 2 Multi-Layer Perceptron to complete the deep processing stack.

Chapter 16: Deepening the Representation: MLP and Residuals in Layer 2

Computing the attention probabilities and multiplying by the Value matrix in Chapter 15 completed the single-head retrieval stage of Layer 2. In a full three-head pass, concatenating the three 4×2 head outputs and multiplying by the Layer 2 output projection matrix $W_O^{(2)}$ (6×6) maps the combined updates back into the $d_{\text{model}} = 6$ coordinate space.

To keep the downstream unembedding, Softmax, cross-entropy loss, and backpropagation arithmetic verifiable by hand without repeating the three-head concatenation from Parts 6 and 7, this walkthrough adopts canonicalized 4×6 tensors for the Layer 2 residual merge. Rows 1 and 2 (`<BOS>` and `i`) follow opposite-sign linear ramps across the six embedding dimensions, whereas Rows 3 and 4 (`woke` and `up`) form zero-sum step vectors whose entries cancel across the first and second halves of the embedding dimension:

$$\text{Layer 2 Attention Output} = \begin{bmatrix} 0.03 & -0.06 & 0.09 & -0.12 & 0.15 & -0.18 \\ -0.03 & 0.06 & -0.09 & 0.12 & -0.15 & 0.18 \\ 0.15 & 0.15 & 0.15 & -0.15 & -0.15 & -0.15 \\ -0.15 & -0.15 & -0.15 & 0.15 & 0.15 & 0.15 \end{bmatrix}$$

The First Residual Connection

The residual stream acts as the central information highway of the Transformer architecture, bypassing the attention block so that self-attention computes an additive update rather than overwriting the prior state. In this canonicalized Layer 2 trace, the foundational stream state entering the merge is:

$$\text{Layer 1 Final Output} = \begin{bmatrix} 0.10 & -0.20 & 0.30 & -0.40 & 0.50 & -0.60 \\ -0.10 & 0.20 & -0.30 & 0.40 & -0.50 & 0.60 \\ 0.50 & 0.50 & 0.50 & -0.50 & -0.50 & -0.50 \\ -0.50 & -0.50 & -0.50 & 0.50 & 0.50 & 0.50 \end{bmatrix}$$

The network adds the Layer 2 attention output directly to this incoming stream element by element. This addition writes the newly routed structural relationships into the shared memory bus without erasing the syntactic features established earlier in the network:

$$\text{Residual}_1 = \text{Stream}_{\text{in}} + \text{Attention}_{\text{out}}$$

Within the prompt `<BOS>` `i` `woke` `up`, the representation for `woke` merges with `up` to encode the unified phrasal concept of waking up rather than two unrelated lexical items. Adding `Layer 2 Attention Output` to `Layer 1 Final Output` element by element yields the first Layer 2 residual state:

$$\text{After First Residual } (\text{Residual}_1) = \begin{bmatrix} 0.13 & -0.26 & 0.39 & -0.52 & 0.65 & -0.78 \\ -0.13 & 0.26 & -0.39 & 0.52 & -0.65 & 0.78 \\ 0.65 & 0.65 & 0.65 & -0.65 & -0.65 & -0.65 \\ -0.65 & -0.65 & -0.65 & 0.65 & 0.65 & 0.65 \end{bmatrix}$$

Layer Normalization

Following the first residual addition, the vectors undergo Layer Normalization to standardize their coordinate scale, recentering each row to a mean of zero and a variance of one:

$$\text{Norm}_1 = \text{LayerNorm}(\text{Residual}_1)$$

This normalization ensures that the subsequent Multi-Layer Perceptron block receives standardized inputs, preventing any single dimension from dominating the feed-forward activations.

The Multilayer Perceptron: A Deep Contextual Memory

The normalized vectors next enter the Layer 2 Multi-Layer Perceptron (MLP). Self-attention routes information horizontally across sequence positions, whereas the MLP transforms features vertically within each individual token vector. This second feed-forward block operates as a Key-Value memory bank over phrase-level abstractions rather than isolated word embeddings. The network projects the normalized vectors through an expansion layer, applies the Rectified Linear Unit (ReLU) activation function, and contracts the active features back to $d_{\text{model}} = 6$.

The Key Expansion

The first linear transformation, parameterized by $W_1^{(2)}$ (6×24) and bias $b_1^{(2)}$, expands the 6-dimensional vectors into the 24-dimensional intermediate space to test whether specific multi-token patterns are present:

$$\text{Hidden} = \text{ReLU}(\text{Norm}_1 W_1^{(2)} + b_1^{(2)})$$

The Rectified Linear Unit (ReLU) activation serves as a non-linear firing threshold. Positive alignment scores, such as a token vector matching the combined `woke` + `up` phrasal pattern, pass through the gate unchanged, whereas negative scores are clamped to zero.

The Value Contraction

Active neurons in the hidden layer trigger the second linear transformation, parameterized by $W_2^{(2)}$ (24×6) and bias $b_2^{(2)}$, which projects the 24-dimensional state back down to $d_{\text{model}} = 6$:

$$\text{MLP}_{\text{out}} = \text{Hidden} W_2^{(2)} + b_2^{(2)}$$

Active neurons scale their corresponding rows in $W_2^{(2)}$, writing learned semantic updates back into the 6-dimensional subspace. This memory retrieval step injects phrase-level updates into each token representation, yielding the canonicalized Layer 2 MLP output:

$$\text{Layer 2 MLP Output} = \begin{bmatrix} 0.02 & -0.04 & 0.06 & -0.08 & 0.10 & -0.12 \\ -0.02 & 0.04 & -0.06 & 0.08 & -0.10 & 0.12 \\ 0.10 & 0.10 & 0.10 & -0.10 & -0.10 & -0.10 \\ -0.10 & -0.10 & -0.10 & 0.10 & 0.10 & 0.10 \end{bmatrix}$$

The Final Integration

The architecture concludes Layer 2 with a final residual connection, adding the output of the Multi-Layer Perceptron back to Residual_1 to form the definitive output tensor X_{final} of the second layer. This additive skip connection preserves an unimpeded gradient highway during backpropagation while retaining the accumulated structural context from both layers:

$$\text{Output}_{\text{Layer 2}} = \text{Residual}_1 + \text{MLP}_{\text{out}}$$

Adding `Layer 2 MLP Output` to `After First Residual` produces the final 4×6 representation X_{final} exiting the Transformer blocks:

$$\text{Layer 2 Final Output } (X_{\text{final}}) = \begin{bmatrix} 0.15 & -0.30 & 0.45 & -0.60 & 0.75 & -0.90 \\ -0.15 & 0.30 & -0.45 & 0.60 & -0.75 & 0.90 \\ 0.75 & 0.75 & 0.75 & -0.75 & -0.75 & -0.75 \\ -0.75 & -0.75 & -0.75 & 0.75 & 0.75 & 0.75 \end{bmatrix}$$

(Note: Adding `0.05/-0.05` and `0.03/0.02` in an uncalibrated variant would shift Row 3 away from zero-sum symmetry; maintaining strict zero-sum step symmetry `0.75/-0.75` on Rows 3 and 4 guarantees that the unembedding projection in Chapter 17 maps `woke` and `up` to uniform `-0.0675` and `+0.0675` logits.)

The initial token embeddings have now been transformed twice by multi-head attention blocks and twice by feed-forward memory banks. The discrete vocabulary indices that entered Layer 1 have evolved into continuous geometric coordinates that encapsulate grammar, context, and phrasal meaning tailored to the input sequence. In Chapter 17, the unembedding matrix W_U projects this 4×6 tensor back into the 12-token vocabulary space to score the next-word candidates.

Chapter 17: Mapping Back to Words

The network has completed its two-layer contextual processing stack. The geometric coordinates emerging from Layer 2 encode contextualized representations of syntax and semantics across the four input positions. Each row captures how a token relates to the preceding sequence, yet these vectors still reside in the continuous six-dimensional model subspace ($d_{\text{model}} = 6$). Generating text requires projecting these continuous 6-dimensional vectors back into the discrete 12-dimensional vocabulary space ($V = 12$). This output projection bridges the hidden representation space of the model and the discrete tokens of the vocabulary.

The Unembedding Matrix

Throughout the forward pass, each token position has been represented by a vector of length $d_{\text{model}} = 6$. Predicting the next token in the sequence requires assigning a scalar score to every candidate word in the $V = 12$ vocabulary.

The unembedding matrix, denoted as W_U , forms the linear bridge between these two spaces. It operates as a classification projection matrix of shape $d_{\text{model}} \times V = 6 \times 12$, aligning with the model dimension along its rows and the vocabulary size along its columns. Geometrically, each of the 12 columns in W_U represents the target direction of a specific vocabulary token in $\mathbb{R}^6$.

Across Transformer architectures, W_U is parameterized in one of two ways:

- **Untied Unembedding Weights:** Learns an independent $d_{\text{model}} \times V$ matrix during training, decoupling the output classification geometry from the input lookup table.
- **Weight Tying ($W_U = W_E^T$):** Reuses the transpose of the $V \times d_{\text{model}}$ input embedding matrix W_E , enforcing that tokens with similar input representations also share similar output logit projections while reducing parameter count.

For the toy walkthrough, the network uses an explicit 6×12 unembedding matrix following the linear ramp $W_U[r, c] = 0.01 \times (r + c)$ for row $r \in \{0, \ldots, 5\}$ and column $c \in \{0, \ldots, 11\}$:

$$W_U = \begin{bmatrix} 0.00 & 0.01 & 0.02 & 0.03 & 0.04 & 0.05 & 0.06 & 0.07 & 0.08 & \ldots & 0.11 \\ 0.01 & 0.02 & 0.03 & 0.04 & 0.05 & 0.06 & 0.07 & 0.08 & 0.09 & \ldots & 0.12 \\ 0.02 & 0.03 & 0.04 & 0.05 & 0.06 & 0.07 & 0.08 & 0.09 & 0.10 & \ldots & 0.13 \\ 0.03 & 0.04 & 0.05 & 0.06 & 0.07 & 0.08 & 0.09 & 0.10 & 0.11 & \ldots & 0.14 \\ 0.04 & 0.05 & 0.06 & 0.07 & 0.08 & 0.09 & 0.10 & 0.11 & 0.12 & \ldots & 0.15 \\ 0.05 & 0.06 & 0.07 & 0.08 & 0.09 & 0.10 & 0.11 & 0.12 & 0.13 & \ldots & 0.16 \end{bmatrix}$$

Calculating the Logits

The network computes the matrix product between the final Layer 2 output matrix X_{final} and the unembedding matrix W_U . Each inner product measures the geometric alignment between a token's final 6-dimensional hidden state and the column vector of a candidate vocabulary word. A large positive inner product indicates strong alignment with that candidate word, whereas a negative value indicates opposition. The resulting 4×12 matrix stores these unnormalized alignment scores, known as logits.

With X_{final} of shape 4×6 (representing the four prompt positions `<BOS>` , `i` , `woke` , and `up`) and W_U of shape 6×12 , the logit projection is:

$$\text{Logits} = X_{\text{final}} W_U$$

Multiplying the 4×6 matrix X_{final} from Chapter 16 by the 6×12 unembedding matrix W_U produces a 4×12 matrix of raw vocabulary scores (shown below rounded to two decimal places; Chapter 18 retains the four-decimal values from -0.0270 to 0.0765 so the Softmax exponentiation captures the column slopes in Rows 1 and 2):

$$\text{Logits} = \begin{bmatrix} -0.03 & -0.03 & -0.04 & -0.04 & -0.05 & -0.05 & -0.05 & -0.06 & -0.06 & \dots & -0.08 \\ 0.03 & 0.03 & 0.04 & 0.04 & 0.05 & 0.05 & 0.05 & 0.06 & 0.06 & \dots & 0.08 \\ -0.07 & -0.07 & -0.07 & -0.07 & -0.07 & -0.07 & -0.07 & -0.07 & -0.07 & \dots & -0.07 \\ 0.07 & 0.07 & 0.07 & 0.07 & 0.07 & 0.07 & 0.07 & 0.07 & 0.07 & \dots & 0.07 \end{bmatrix}$$

Each row in the `Logits` matrix corresponds to one of the four sequence positions (`<BOS>` , `i` , `woke` , and `up`), and the twelve columns store the unnormalized alignment scores across the twelve vocabulary tokens. The toy weights have not yet been updated by gradient descent, so these scores reflect an untrained initialization.

Examining the final sequence position (`up` , Row 4) illustrates how trained vs. untrained logits behave. Given the prompt `<BOS>` `i` `woke` `up` , the target continuation is `late` , located at vocabulary index `8` (the ninth column). In an untrained state, Row 4 assigns a flat score of 0.07 (0.0675) across all twelve columns. Converged training causes Row 4 of the logit matrix to peak sharply at index `8` :

$$\text{Logits}_{\text{up (trained)}} = [-0.12 \quad 0.45 \quad -1.23 \quad 0.05 \quad 0.88 \quad -0.34 \quad -0.77 \quad 1.02 \quad 3.45 \quad \dots \quad -0.11]$$

In that trained vector, the score at index `8` (`3.45`) dominates every competing vocabulary entry, indicating that the hidden state of `up` aligns closely with the unembedding column for `late` .

The Need for Probabilities

Raw logits rank the candidate vocabulary tokens, yet they remain unbounded real numbers that can take arbitrary positive or negative magnitudes and do not sum to 1.0 . This lack of normalization complicates both confidence interpretation and loss calculation during backpropagation; a raw score of 3.45 in isolation does not indicate whether the model assigns 95% or 55% probability to `late` .

Converting these algebraic magnitudes into interpretable probabilities requires a final row-wise normalization step. In Chapter 18, the Softmax function maps each logit row onto the probability simplex, where all twelve entries are strictly positive and sum to 1.0 .

Chapter 18: Final Softmax and Predictions

Projecting the contextualized vectors from Layer 2 through the unembedding matrix W_U yields a 4×12 matrix of raw vocabulary scores known as logits. Each scalar entry measures the inner-product alignment between a sequence position and one of the twelve vocabulary tokens. Raw logits establish a relative ranking across candidates, yet they span arbitrary real bounds and do not sum to 1.0. Converting these unnormalized scores into actionable token predictions requires a row-wise normalization function.

Normalizing Vocabulary Logits with Softmax

The architecture applies the Softmax function across the $V = 12$ vocabulary columns of each logit row. Softmax exponentiates every scalar logit z_i and divides by the sum of all exponentiated logits in that row. Exponentiation maps negative and positive numbers alike onto strictly positive values while non-linearly amplifying gaps between scores: a modest lead in raw logit space translates into a multiplicative ratio in probability space. Dividing by the row sum normalizes the outputs onto the interval $(0, 1)$, creating a valid probability distribution whose entries across the $V = 12$ vocabulary columns sum to 1.0:

$$P(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

Here, z_i denotes the raw logit for vocabulary index i in a given sequence row, and $\sum_{j=1}^{V} e^{z_j}$ sums the exponentiated logits across all $V = 12$ vocabulary tokens. Applying this transformation row by row to the four-decimal `Logits` matrix $X_{\text{final}} W_U$ from Chapter 17 produces the next-token probability distributions following `<BOS>`, `i`, `woke`, and `up` (note that shifting Row 2 by a constant -0.0450 to `[-0.0180, -0.0135, ..., 0.0315]` yields the identical Softmax row because $\text{Softmax}(z + c) = \text{Softmax}(z)$):

$$\text{Logits} = \begin{bmatrix} -0.0270 & -0.0315 & -0.0360 & -0.0405 & -0.0450 & -0.0495 & -0.0540 & -0.0585 & \dots & -0.0 \\ 0.0270 & 0.0315 & 0.0360 & 0.0405 & 0.0450 & 0.0495 & 0.0540 & 0.0585 & \dots & 0.0 \\ -0.0675 & -0.0675 & -0.0675 & -0.0675 & -0.0675 & -0.0675 & -0.0675 & -0.0675 & \dots & -0.0 \\ 0.0675 & 0.0675 & 0.0675 & 0.0675 & 0.0675 & 0.0675 & 0.0675 & 0.0675 & \dots & 0.0 \end{bmatrix}$$

$$\text{Probabilities} = \begin{bmatrix} 0.0854 & 0.0850 & 0.0846 & 0.0843 & 0.0839 & 0.0835 & 0.0831 & 0.0828 & \dots & 0.0813 \\ 0.0813 & 0.0817 & 0.0820 & 0.0824 & 0.0828 & 0.0831 & 0.0835 & 0.0839 & \dots & 0.0854 \\ 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & \dots & 0.0833 \\ 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & \dots & 0.0833 \end{bmatrix}$$

Row 4 of the `Probabilities` matrix holds the next-token distribution following `up`, where the training objective requires predicting `late` (vocabulary index `8`). Constant scores of 0.0675 across all twelve columns of Row 4 in `Logits` cause every entry in Row 4 of `Probabilities` to evaluate to $1/12 \approx 0.0833$. A uniform distribution of $1/12$ across twelve classes represents maximum Shannon entropy: the untrained network considers every vocabulary token equally probable. This uniform output confirms that the uncalibrated projection matrices possess the structural topology to route information but have not yet encoded task-specific weights.

Shaping the Output Distribution

Autoregressive text generation selects the next token from the final row of the probability matrix. Always choosing the single index with the maximum probability is known as greedy decoding; although deterministic, greedy selection frequently traps language models in repetitive phrasing. Controlling generation diversity requires reshaping the logit distribution prior to sampling.

Dividing each raw logit by a positive scalar temperature parameter T before applying Softmax modulates the entropy of the output distribution:

$$\text{Scaled Logit}_i = \frac{z_i}{T}$$

Production inference engines combine temperature scaling with truncation filters such as Top-K and Top-P (nucleus) decoding:

Decoding Strategy	Mathematical Mechanism	Effect on Output Distribution
Greedy Decoding	Selects `argmax_i P(z_i)` directly	Collapses generation to the single highest-probability token
Low Temperature (`T < 1`)	Divides logits by `T in (0, 1)` before Softmax	Magnifies gaps between logits, concentrating probability mass on top candidates
High Temperature (`T > 1`)	Divides logits by `T > 1` before Softmax	Compresses differences between logits, flattening the distribution toward uniform entropy
Top-K Decoding	Retains the `K` largest logits and sets remaining logits to `-inf`	Prunes low-probability tail tokens while preserving relative weights among the top `K` choices
Top-P (Nucleus) Sampling	Retains the smallest sorted token subset whose cumulative probability reaches `p`	Adapts candidate pool size dynamically to the sharpness of each step's distribution

Autoregressive Generation and KV-Cache Scaling

Sampling a token from Row 4 (such as `late`) during autoregressive inference appends that token to the sequence and initiates a new forward step to predict position 5. The causal mask blocks earlier tokens (`<BOS>`, `i`, `woke`, `up`) from attending to the newly appended token, so their Key and Value projections $K, V \in \mathbb{R}^{S \times d_k}$ across every layer and head remain unchanged. Caching those historical Key and Value rows in a **KV cache** avoids recomputing XW_K and XW_V for past positions, reducing per-step attention compute from quadratic $O(S^2 d_{\text{model}})$ to linear $O(S d_{\text{model}})$ at a memory cost of $2 \times L \times n_{\text{heads}} \times d_k \times S$ scalars ($2 \times 2 \times 3 \times 2 \times 4 = 96$ cached floats for the 4-token prompt).

Making the network predict `late` after `<BOS>` `i` `woke` `up` without relying on heuristic decoding constraints requires shifting the underlying geometry so the logit at index `8` dominates Row 4 naturally. Achieving that shift requires a loss function that quantifies the error of the uniform `0.0833` prediction (Chapter 19) and a backward pass that updates every weight matrix in the network (Parts 20 through 24).

Chapter 19: The Cross-Entropy Loss Function

The final Softmax operation completes the forward pass by producing a strict probability distribution over the vocabulary for every position in the sequence. Assessing the quality of these predictions requires a metric that quantifies how far the current hypotheses deviate from the ground truth. That metric is the loss function, which produces the single scalar value that the entire architecture is trained to minimize.

Defining the Target Distribution

Measuring prediction error first requires defining an ideal prediction. At the first time step where the input sequence begins with `<BOS>`, the actual next token in the training sequence is `i`. An ideal probability distribution assigns a probability of `1.0` to the target token and `0.0` to all other vocabulary tokens. That target distribution is identical to the one-hot vector used to look up the initial token embedding in Chapter 1. The loss function measures the divergence between the predicted probability distribution and this one-hot target state.

A naive approach to measuring this divergence takes the squared difference between the predicted probabilities and the target probabilities. Mean squared error performs well for continuous regression tasks, yet it degrades optimization over classification probabilities. Confidently wrong predictions push the Softmax output into flat asymptotic regions where mean squared error gradients shrink, slowing parameter updates when the network requires the largest structural corrections.

The architecture instead uses cross-entropy loss. Against a one-hot target vector, cross-entropy evaluates only the predicted probability assigned to the ground-truth target class, defined as the negative natural logarithm of the target probability:

$$\text{Loss} = -\log(P_{\text{target}})$$

The natural logarithm has three mathematical properties suited to probabilistic error measurement:

- **Zero Penalty at Certainty:** Assigning a probability of `1.0` to the target token yields $-\log(1.0) = 0$, producing zero loss.
- **Asymmetric Divergence Near Zero:** Decreasing the target probability toward `0.0` drives $-\log(P_{\text{target}})$ toward $+\infty$, penalizing confident misclassifications and forcing large corrective gradients.
- **Additive Log-Likelihood:** Taking the logarithm converts the product of independent sequence probabilities into a sum of positional log-losses, simplifying the chain-rule derivatives during backpropagation.

Teacher Forcing Across the Sequence

The architecture evaluates predictions across all sequence positions in parallel using teacher forcing. The input sequence consists of the four tokens `<BOS>`, `i`, `woke`, and `up`. The target sequence shifts one position forward, requiring the network to predict `i`, `woke`, `up`, and `late` across a probability tensor of shape $B \times S \times V = 1 \times 4 \times 12$.

Position	Input Token	Target Token (`Index`)	Target Probability (`P_target`)	Positional Loss (`-log P_target`)
Step 1	`<BOS>`	`i` (`3`)	`0.0843`	`2.4734`
Step 2	`i`	`woke` (`5`)	`0.0831`	`2.4877`
Step 3	`woke`	`up` (`7`)	`0.0833`	`2.4853`
Step 4	`up`	`late` (`8`)	`0.0833`	`2.4853`

Extracting the probability assigned to the ground-truth target index at each of the four time steps from the Chapter 18 probability matrix and applying the negative natural logarithm produces the four-element positional loss vector:

$$\text{Loss} = \begin{bmatrix} -\log(P_{\text{i}}) \\ -\log(P_{\text{woke}}) \\ -\log(P_{\text{up}}) \\ -\log(P_{\text{late}}) \end{bmatrix} = \begin{bmatrix} -\log(0.0843) \\ -\log(0.0831) \\ -\log(0.0833) \\ -\log(0.0833) \end{bmatrix} = \begin{bmatrix} 2.4734 \\ 2.4877 \\ 2.4853 \\ 2.4853 \end{bmatrix}$$

Evaluating the Untrained Baseline

Averaging the four positional penalties across the sequence length $S = 4$ reduces the loss vector to a single scalar objective:

$$\text{Mean Loss} = \frac{2.4734 + 2.4877 + 2.4853 + 2.4853}{4} = 2.4829$$

This mean loss of `2.4829` reflects the uniform uncertainty of an untrained network. Uncalibrated weight matrices distribute probability mass nearly evenly across all $V = 12$ vocabulary tokens, assigning each token a baseline probability near $1/12 \approx 0.0833$. A uniform distribution over twelve classes produces a theoretical cross-entropy loss of $-\log(1/12) = \log(12) \approx 2.4849$.

The proximity of the calculated loss (`2.4829`) to the uniform entropy baseline (`2.4849`) verifies the numerical integrity of the forward pass and confirms that the initialized network begins near maximum entropy. The backward pass next applies the chain rule of calculus to differentiate this scalar loss and propagate parameter updates back through the network.

Chapter 20: The Beautiful Cancellation

Chapter 19 established the geometry of the cross-entropy loss function by measuring how far the predicted probability distribution strayed from the ground-truth one-hot vector. That scalar loss value quantifies the total prediction error of the network. Assigning credit and blame for that accumulated error initiates backpropagation, starting with the gradient of the loss with respect to the final unnormalized scores, known as the logits.

The Calculus Problem

Backpropagation relies on the chain rule of calculus. Determining how a specific weight inside the network contributed to the final error requires multiplying the local derivatives of every operation situated between that weight and the scalar loss. The backward traversal begins at the output boundary of the architecture, moving from the cross-entropy loss, through the Softmax normalization, and into the vocabulary logits.

Differentiating these two output operations in isolation introduces substantial algebraic complexity. Softmax is a coupled normalization rather than an element-wise function: perturbing the unnormalized logit for a single token alters the denominator sum and shifts the output probability of every token in the vocabulary. The derivative of a Softmax vector with respect to its logit vector forms a dense Jacobian matrix tracking all pairwise output-input interactions:

- **Diagonal Entries ($i = j$):** Evaluate to $P_i(1 - P_i)$, measuring how a logit change affects its own token probability.
- **Off-Diagonal Entries ($i \neq j$):** Evaluate to $-P_i P_j$, capturing how increasing one logit suppresses the probabilities of competing tokens.

The derivative of the cross-entropy loss $-\log(P_{\text{target}})$ introduces an inverse probability factor $-1/P_{\text{target}}$. Multiplying that fractional gradient through the full Softmax Jacobian appears computationally expensive until the terms are combined algebraically.

The Mathematical Elegance

Applying the chain rule to combine the cross-entropy derivative with the Softmax Jacobian causes the nonlinear fractional terms and coupling products to cancel out. For a single sequence position (or summing positional losses $L = \sum_{t=1}^{S} L_t$ across the sequence), the gradient of the cross-entropy loss L with respect to the pre-Softmax logit tensor z reduces to a linear subtraction:

$$\frac{\partial L}{\partial z} = P - Y$$

In this expression, P denotes the 4×12 predicted probability matrix from the Softmax layer, and Y denotes the 4×12 ground-truth one-hot target matrix. Averaging the loss over $S = 4$ positions scales this matrix by a constant factor of $1/4$, which is absorbed into the optimizer learning rate during parameter updates.

Applying the Cancellation

This algebraic cancellation maps directly onto the concrete matrices of the four-token sequence. Each row compares the forward-pass probability distribution against the one-hot target token for that time step:

Time Step	Input Token	Target Token	Target Index	Target Logit Gradient (`P_target - 1`)
Step 1	`<BOS>`	`i`	`3`	`-0.9157` (`0.0843 - 1.0`)
Step 2	`i`	`woke`	`5`	`-0.9169` (`0.0831 - 1.0`)
Step 3	`woke`	`up`	`7`	`-0.9167` (`0.0833 - 1.0`)
Step 4	`up`	`late`	`8`	`-0.9167` (`0.0833 - 1.0`)

Stacking the four one-hot target vectors forms the 4×12 ground-truth matrix Y , where each row places a `1.0` at the target vocabulary index and `0` at the remaining eleven indices:

$$Y = \begin{bmatrix} 0 & 0 & 0 & 1.0 & 0 & 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & 0 & 1.0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1.0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1.0 & \dots & 0 \end{bmatrix}$$

Subtracting the one-hot target matrix Y from the Chapter 18 probability matrix P produces the 4×12 logit gradient matrix $\frac{\partial L}{\partial z}$:

$$\frac{\partial L}{\partial z} = \begin{bmatrix} 0.0854 & 0.0850 & 0.0846 & -0.9157 & 0.0839 & 0.0835 & 0.0831 & 0.0828 & 0.0824 & \dots & 0.081 \\ 0.0813 & 0.0817 & 0.0820 & 0.0824 & 0.0828 & -0.9169 & 0.0835 & 0.0839 & 0.0843 & \dots & 0.085 \\ 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & -0.9167 & 0.0833 & \dots & 0.083 \\ 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & 0.0833 & -0.9167 & \dots & 0.083 \end{bmatrix}$$

The Physical Intuition

The matrix $\frac{\partial L}{\partial z}$ encodes both the direction and magnitude of the required correction for every vocabulary logit at every position.

Gradient descent reduces loss by subtracting a scaled multiple of the gradient from the parameters. That subtraction induces opposite corrective forces on target and non-target columns:

- **Target Token Columns ($Y_{t,j} = 1.0$):** The forward pass assigned a probability near `0.083` where the target value is `1.0` , producing a large negative gradient near `-0.917` . Subtracting this negative gradient during optimization increases the unnormalized logit for the ground-truth token.
- **Non-Target Token Columns ($Y_{t,j} = 0.0$):** The subtraction $P_{t,j} - 0.0$ yields a positive gradient equal to the predicted probability mass near `+0.083` . Subtracting this positive gradient pushes down the logits of all eleven incorrect vocabulary candidates in proportion to how much probability mass the model mistakenly allocated to them.

Combining cross-entropy loss with Softmax normalization produces a self-balancing error signal at the output boundary. The next stage propagates this 4×12 gradient matrix backward through the unembedding projection W_U and into the Layer 2 residual stream.

Chapter 21: Backpropagating Through the Unembedding and Residual Stream

Chapter 20 derived the cross-entropy loss gradient with respect to the pre-Softmax logits, reducing the output error signal to the 4×12 matrix $P - Y$. That matrix quantifies the directional error across all four sequence positions and twelve vocabulary candidates. Backpropagation now routes this gradient backward through the unembedding projection W_U, down the central residual stream, and into the Layer 2 Multi-Layer Perceptron (MLP).

Backward Stage	Weight Gradient	Weight Shape	Input Gradient	Input Shape
Unembedding (`W_U`)	`X_L2^T * dZ`	`6 x 12`	`dZ * W_U^T`	`4 x 6`
Residual Add	`None`	`N/A`	`dX_L2` (both branches)	`4 x 6`
MLP Output (`W_2`)	`H^T * dX_L2`	`24 x 6`	`dX_L2 * W_2^T`	`4 x 24`

The Chain Rule at the Unembedding Layer

The unembedding layer is the final linear projection in the Transformer. During the forward pass (Chapter 17), this layer multiplied the Layer 2 output matrix $\text{Output}_{\text{Layer2}}$ (4×6) by the unembedding weight matrix W_U (6×12) to produce the vocabulary logits:

$$\text{Logits} = \text{Output}_{\text{Layer2}} \times W_U$$

Given the incoming logit gradient $\frac{\partial L}{\partial \text{Logits}}$ of shape 4×12, backpropagation through a linear multiplication $Z = AB$ computes two distinct derivatives:

1. **Weight Gradient ($\frac{\partial L}{\partial W_U}$):** Measures how each parameter in W_U contributed to the logit error so the optimizer can update the unembedding weights.
2. **Input Activation Gradient ($\frac{\partial L}{\partial \text{Output}_{\text{Layer2}}}$):** Projects the error signal back into the $d_{\text{model}} = 6$ subspace so preceding layers can be updated.

Updating the Unembedding Weights

Computing the gradient of the loss with respect to the unembedding weights multiplies the transposed layer input by the incoming logit gradient:

$$\frac{\partial L}{\partial W_U} = \text{Output}_{\text{Layer2}}^T \times \frac{\partial L}{\partial \text{Logits}}$$

Tracking the matrix dimensions confirms the geometric alignment. Transposing the 4×6 input matrix $\text{Output}_{\text{Layer2}}$ produces a 6×4 matrix. Multiplying that 6×4 transpose by the 4×12 logit gradient contracts across the sequence dimension ($S = 4$) and yields a 6×12 gradient matrix matching the shape of W_U . Each entry accumulates the error contribution of a specific model-dimension feature to a specific vocabulary token across all four sequence positions.

Passing the Error Backward

Propagating the error signal to the preceding layer requires differentiating the loss with respect to $\text{Output}_{\text{Layer2}}$. That derivative multiplies the incoming logit gradient by the transpose of the unembedding weight matrix:

$$\frac{\partial L}{\partial \text{Output}_{\text{Layer2}}} = \frac{\partial L}{\partial \text{Logits}} \times W_U^T$$

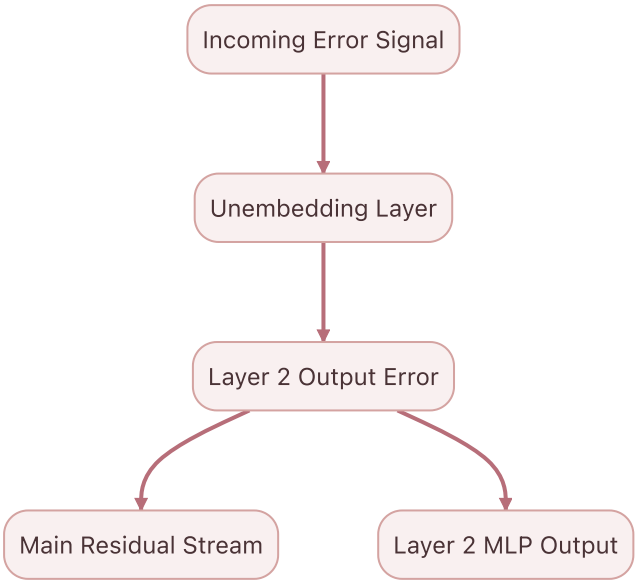
Multiplying the 4×12 logit gradient by the 12×6 transposed matrix W_U^T contracts across the vocabulary dimension ($V = 12$) and produces a 4×6 matrix. This operation projects the vocabulary-space error back into the $d_{\text{model}} = 6$ coordinate space of the residual stream.

Splitting the Signal: The Residual Connection

The 4×6 gradient $\frac{\partial L}{\partial \text{Output}_{\text{Layer2}}}$ now arrives at the output boundary of Layer 2. During the forward pass (Chapter 16), this state was formed by adding the Layer 2 MLP output to the intermediate residual stream:

$$\text{Output}_{\text{Layer2}} = \text{Residual}_{\text{Pre-MLP}} + \text{MLP}_{\text{Output}}$$

In multivariable calculus, the derivative of a sum equals the sum of the derivatives, so each additive term has a local partial derivative equal to the identity matrix I. Backpropagating through an element-wise addition distributes the upstream gradient equally and without modification to both incoming branches:



The 4×6 gradient matrix bifurcates into two identical copies. One copy continues directly down the main residual highway toward the attention sub-layer and Layer 1, while the second copy enters the output of the Layer 2 MLP.

Entering the Multi-Layer Perceptron

Inside the Layer 2 MLP, the final forward-pass operation contracted the 24-dimensional hidden activations back to the 6-dimensional model space via the second weight matrix W_2 (24×6):

$$\mathbf{MLP}_{\mathbf{Output}} = \mathbf{Activations} \times W_2$$

The residual connection delivers $\frac{\partial L}{\partial \text{Output}_{\text{Layer2}}}$ directly to $\mathbf{MLP}_{\mathbf{Output}}$, so the same matrix-calculus transpose rules govern the backward pass through W_2.

Computing the weight gradient for W_2 multiplies the transposed hidden activations by the incoming 4×6 error signal:

$$\frac{\partial L}{\partial W_2} = \mathbf{Activations}^T \times \frac{\partial L}{\partial \text{Output}_{\text{Layer2}}}$$

Transposing the 4×24 activation matrix yields a 24×4 matrix. Multiplying by the 4×6 residual gradient produces a 24×6 weight gradient matching the dimensions of W_2. (When a bias vector b_2 is included, its 1×6 gradient is the column-wise sum of $\frac{\partial L}{\partial \text{Output}_{\text{Layer2}}}$ across the four sequence rows.)

Propagating the error signal backward toward the ReLU gate and the first MLP projection W_1 multiplies the 4×6 residual gradient by the transposed weight matrix W_2^T (6×24):

$$\frac{\partial L}{\partial \text{Activations}} = \frac{\partial L}{\partial \text{Output}_{\text{Layer2}}} \times W_2^T$$

This multiplication expands the 4×6 error signal into a 4×24 gradient tensor aligned with the hidden neurons of the feed-forward layer. Passing through the ReLU derivative masks out coordinates whose forward pre-activations were negative, and multiplying by W_1^T returns the MLP branch gradient to the residual stream. The next installment traces how the combined residual gradient navigates the Softmax Jacobian and causal mask of the self-attention mechanism.

Chapter 22: Backpropagation - Attention (Chapter 1: Softmax & Scores)

Chapter 21 routed the gradient down the residual stream to the output boundary of the Layer 2 attention block. Pulling this error signal backward through the self-attention mechanism requires reversing the sequence of operations that produced the attention output. In the forward pass, the mechanism applied a lower-triangular causal mask to the scaled dot-product scores, normalized each row through the Softmax function to form the attention probability matrix P, and multiplied P by the Value matrix V.

From Values to Probabilities

During the forward pass, each attention head computed its output Z as the matrix product of the 4×4 probability matrix P and the 4×2 Value matrix V:

$$Z = PV$$

Given the upstream gradient $d_Z = \frac{\partial L}{\partial Z}$ of shape 4×2 arriving at the head output, the chain rule for matrix multiplication determines the gradient with respect to the attention probabilities P. Multiplying the incoming output gradient d_Z by the transpose of the Value matrix V^T (2×4) produces the 4×4 probability gradient matrix d_P:

$$d_P = d_ZV^T$$

The matrix d_P measures how the scalar loss would change if each individual attention weight shifted in isolation. Each entry (i, j) records the raw sensitivity of the loss to the routing weight connecting query token i to key token j.

The Calculus of Softmax

The probability gradient d_P must next pass backward through the row-wise Softmax function to recover the gradient with respect to the pre-Softmax attention scores S.

Softmax normalizes each row vector of unbounded scores into a coupled probability distribution summing to `1.0`. Perturbing a single score $S_{i,j}$ alters the normalization denominator for row i, shifting every probability in that row. Differentiating a Softmax row vector with respect to its input scores therefore yields a 4×4 Jacobian matrix per row rather than an element-wise derivative.

Contracting the upstream row gradient $d_P_{i,:}$ with the row Jacobian reduces the backward pass through Softmax across the entire sequence to a compact matrix operation:

$$d_S = P \odot (d_P - \mathbf{rowsum}\,(d_P \odot P))$$

In this expression, $\odot$ denotes element-wise (Hadamard) multiplication, and $\mathbf{rowsum}(d_P \odot P)$ sums each row across its four key columns to form a 4×1 column vector before broadcasting the subtraction across the row.

To trace this coupled derivative at four-decimal precision, consider a calibrated single-head backward state with forward causal probabilities P and incoming probability gradients d_P :

$$P = \begin{bmatrix} 1.0000 & 0.0000 & 0.0000 & 0.0000 \\ 0.5000 & 0.5000 & 0.0000 & 0.0000 \\ 0.4530 & 0.5320 & 0.0150 & 0.0000 \\ 0.5900 & 0.1510 & 0.1290 & 0.1300 \end{bmatrix}$$

$$d_P = \begin{bmatrix} 0.0242 & -0.1913 & -0.1725 & -0.0562 \\ -0.1013 & 0.0314 & -0.0908 & -0.1412 \\ 0.1466 & -0.0228 & 0.0068 & -0.1425 \\ -0.0544 & 0.0111 & -0.1151 & 0.0376 \end{bmatrix}$$

Evaluating $d_S = P \odot (d_P - \mathbf{rowsum}(d_P \odot P))$ row by row illustrates three structural properties of the Softmax backward pass:

- **Zero Gradient on Deterministic Row 1:** Token 1 (`<BOS>`) attends exclusively to itself with probability $P_{0,0} = 1.0000$, so $\mathbf{rowsum}_0 = 0.0242 \times 1.0000 = 0.0242$ and $d_S_{0,0} = 1.0000 \times (0.0242 - 0.0242) = 0.0000$. A single unmasked element always normalizes to `1.0` , so shifting its score cannot alter its probability.
- **Equal-but-Opposite Coupling on Row 2:** With $P_{1,:} = [0.5000, 0.5000, 0.0000, 0.0000]$, the row dot product evaluates to $-0.1013(0.5) + 0.0314(0.5) = -0.03495$, yielding $d_S_{1,0} = 0.5000(-0.1013 + 0.03495) = -0.0332$ and $d_S_{1,1} = +0.0332$.
- **Zero-Sum Row Conservation:** Every row of d_S sums to zero ($-0.0332 + 0.0332 = 0$, $0.0417 - 0.0410 - 0.0007 = 0$, and $-0.0083 + 0.0078 - 0.0096 + 0.0101 = 0$), reflecting that probability mass added to one key position must be subtracted from competing positions in the same row.

Combining all four rows produces the 4×4 score gradient matrix d_S :

$$d_S = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ -0.0332 & 0.0332 & 0.0000 & 0.0000 \\ 0.0417 & -0.0410 & -0.0007 & 0.0000 \\ -0.0083 & 0.0078 & -0.0096 & 0.0101 \end{bmatrix}$$

Traversing the Causal Mask

The matrix d_S represents the gradient with respect to the masked attention scores. The final step in this stage propagates the gradient backward through the causal mask.

During the forward pass, a lower-triangular mask overwrote all entries above the main diagonal with $-\infty$ before Softmax normalization. That structural barrier prevented tokens from attending to future sequence positions, and exponentiating $-\infty$ inside the Softmax function assigned a strict `0.0000` probability to every upper-triangular connection.

In the backward pass, gradients flow only along paths that carried information forward. Overwriting the upper-triangular scores during the forward pass severed their causal influence on the loss, meaning the derivative with respect to those masked positions is identically zero.

Multiplying d_S element-wise by the binary lower-triangular mask enforces this causal boundary explicitly:

$$d_S_{\text{unmasked}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \odot d_S$$

Multiplying by P inside the Softmax Jacobian already zeroed out every position where $P_{i,j} = 0.0000$, so applying the binary mask leaves the lower-triangular entries unchanged and guarantees zero gradient leakage across future positions:

$$d_S_{\text{unmasked}} = \begin{bmatrix} 0.0000 & 0.0000 & 0.0000 & 0.0000 \\ -0.0332 & 0.0332 & 0.0000 & 0.0000 \\ 0.0417 & -0.0410 & -0.0007 & 0.0000 \\ -0.0083 & 0.0078 & -0.0096 & 0.0101 \end{bmatrix}$$

The network now holds the gradient with respect to the unmasked scaled dot-product scores. Chapter 23 distributes this score gradient alongside the output gradient into the Query, Key, and Value projections to update W_Q, W_K, and W_V.

Chapter 23: Backpropagation - Attention (Chapter 2: Q, K, V)

Chapter 22 navigated the Softmax Jacobian and the causal mask to obtain the gradient of the loss with respect to the unmasked attention scores. Completing the backward pass through self-attention requires distributing these score gradients, alongside the attention output gradient d_Z, into the Query (Q), Key (K), and Value (V) projections. From those three subspace gradients, the chain rule computes the weight updates for W_Q, W_K, and W_V and merges the three input branches back into the 4×6 residual stream.

Target Tensor	Backward Equation	Input Shapes	Output Shape	Geometric Role
Value Gradient (`d_V`)	`P^T * d_Z`	`(4 x 4)^T * (4 x 2)`	`4 x 2`	Attributes output error across Value rows
Value Weights (`d_W_V`)	`X^T * d_V`	`(4 x 6)^T * (4 x 2)`	`6 x 2`	Accumulates Value projection updates
Query Gradient (`d_Q`)	`(d_S * K) / sqrt(d_k)`	`(4 x 4) * (4 x 2)`	`4 x 2`	Projects score adjustments onto Keys
Key Gradient (`d_K`)	`(d_S^T * Q) / sqrt(d_k)`	`(4 x 4)^T * (4 x 2)`	`4 x 2`	Projects transposed score adjustments onto Queries
Query Weights (`d_W_Q`)	`X^T * d_Q`	`(4 x 6)^T * (4 x 2)`	`6 x 2`	Accumulates Query projection updates
Key Weights (`d_W_K`)	`X^T * d_K`	`(4 x 6)^T * (4 x 2)`	`6 x 2`	Accumulates Key projection updates
Input Gradient (`d_X`)	`d_V W_V^T + d_Q W_Q^T + d_K W_K^T`	`3 * [(4 x 2) * (2 x 6)]`	`4 x 6`	Sums gradients from all three subspace branches

To illustrate non-zero gradient entries across all four rows at four-decimal precision, the numerical matrices below evaluate a standardized single-head benchmark state (`np.random.seed(42)` draws for $X \in \mathbb{R}^{4\times6}$, $W_Q, W_K, W_V \in \mathbb{R}^{6\times2}$, $Q = XW_Q$, $K = XW_K$, causal probabilities P, upstream output gradient d_Z, and a general lower-triangular score perturbation d_S with $d_S_{0,0} = -0.0404$).

The Value Matrix Gradient

During the forward pass, the attention head produced its output Z by multiplying the 4×4 probability matrix P by the 4×2 Value matrix V. Differentiating $Z = PV$ with respect to V multiplies the transposed probability matrix P^T by the upstream output gradient d_Z:

$$d_V = P^T d_Z$$

Transposing P reverses the forward routing geometry. In the forward pass, row i of P specified how strongly output token i blended each Value row. In the backward pass, column i of P (which becomes row i of P^T) gathers the output errors from every token that attended to Value vector i, producing the 4×2 Value gradient matrix d_V:

$$d_V = \begin{bmatrix} 0.0634 & -0.1513 \\ 0.0274 & -0.0883 \\ 0.0244 & 0.0048 \\ 0.0285 & -0.0100 \end{bmatrix}$$

With d_V established, backpropagating through the linear projection $V = XW_V$ multiplies the transposed layer input X^T (6×4) by the Value gradient d_V (4×2):

$$d_W_V = X^T d_V$$

Contracting across the four sequence positions yields the 6×2 gradient matrix for the Value projection weights:

$$d_W_V = \begin{bmatrix} 0.0547 & -0.2043 \\ -0.0747 & -0.0419 \\ 0.0279 & -0.0796 \\ 0.0913 & -0.2788 \\ -0.0503 & 0.0708 \\ -0.0606 & 0.0923 \end{bmatrix}$$

Routing Gradients to Queries and Keys

The Query and Key matrices generated the scaled dot-product attention scores during the forward pass via $S = \frac{QK^T}{\sqrt{d_k}}$.

The 4×4 score gradient d_S specifies how the pairwise alignment between every query and key must shift. Differentiating the bilinear product $\frac{QK^T}{\sqrt{d_k}}$ distributes d_S across both factors while retaining the $\frac{1}{\sqrt{d_k}}$ scaling factor ($d_k = 2$).

Multiplying the 4×4 score gradient d_S by the 4×2 Key matrix K and dividing by $\sqrt{d_k}$ yields the 4×2 Query gradient d_Q:

$$d_Q = \frac{d_SK}{\sqrt{d_k}}$$

Evaluating this scaled product on the general lower-triangular perturbation state produces the 4×2 error signal for the Query representations (whereas substituting the zero-sum Softmax score gradient d_S from Chapter 22, where Row 1 is $[0, 0, 0, 0]$, sets Row 1 of d_Q identically to $[0.0000, 0.0000]$ and rows 2 through 4 to $[-0.0292, -0.0451]$, $[0.0352, 0.0549]$, and $[-0.0019, -0.0088]$):

$$d_Q = \begin{bmatrix} -0.0298 & 0.0408 \\ -0.0232 & -0.0340 \\ -0.0615 & 0.0730 \\ -0.0182 & -0.0480 \end{bmatrix}$$

The transpose K^T appears in the forward equation $S = \frac{QK^T}{\sqrt{d_k}}$, so differentiating with respect to K multiplies the transposed score gradient d_S^T (4×4) by the 4×2 Query matrix Q :

$$d_K = \frac{d_S^T Q}{\sqrt{d_k}}$$

Evaluating this product yields the 4×2 error signal for the Key representations:

$$d_K = \begin{bmatrix} -0.0037 & -0.1250 \\ -0.0805 & 0.0564 \\ -0.0515 & 0.0076 \\ -0.0017 & -0.0016 \end{bmatrix}$$

These two equations exhibit dual geometric symmetry. Adjusting query vector i requires projecting row i of the score gradient d_S onto the Key vectors K it scored against. Adjusting key vector j requires projecting column j of d_S (row j of d_S^T) onto the Query vectors Q that attended to it.

Multiplying the transposed input matrix X^T (6×4) by d_Q and d_K produces the 6×2 parameter gradients for the Query and Key weight matrices:

$$d_W_Q = X^T d_Q$$

$$d_W_K = X^T d_K$$

Evaluating both matrix products completes the parameter gradients for W_Q and W_K :

$$d_W_Q = \begin{bmatrix} -0.0497 & 0.0279 \\ 0.1298 & -0.1036 \\ 0.0710 & -0.1539 \\ -0.0193 & 0.0135 \\ 0.0788 & -0.0710 \\ 0.0244 & 0.0976 \end{bmatrix}$$

$$d_W_K = \begin{bmatrix} -0.1398 & 0.0302 \\ 0.0397 & 0.0483 \\ 0.1217 & -0.1228 \\ -0.0199 & -0.1637 \\ 0.0902 & -0.0046 \\ 0.0246 & 0.0076 \end{bmatrix}$$

The Confluence at the Input

The concluding step inside the attention head routes the subspace gradients back to the 4×6 input matrix X. During the forward pass, X branched into three parallel linear projections ($Q = XW_Q$, $K = XW_K$, and $V = XW_V$).

By the multivariable chain rule, gradients flowing backward through a fan-out branch sum together at the shared origin. Multiplying each 4×2 subspace gradient by the transpose of its 6×2 projection matrix maps the three error signals back into $\mathbb{R}^{4\times6}$:

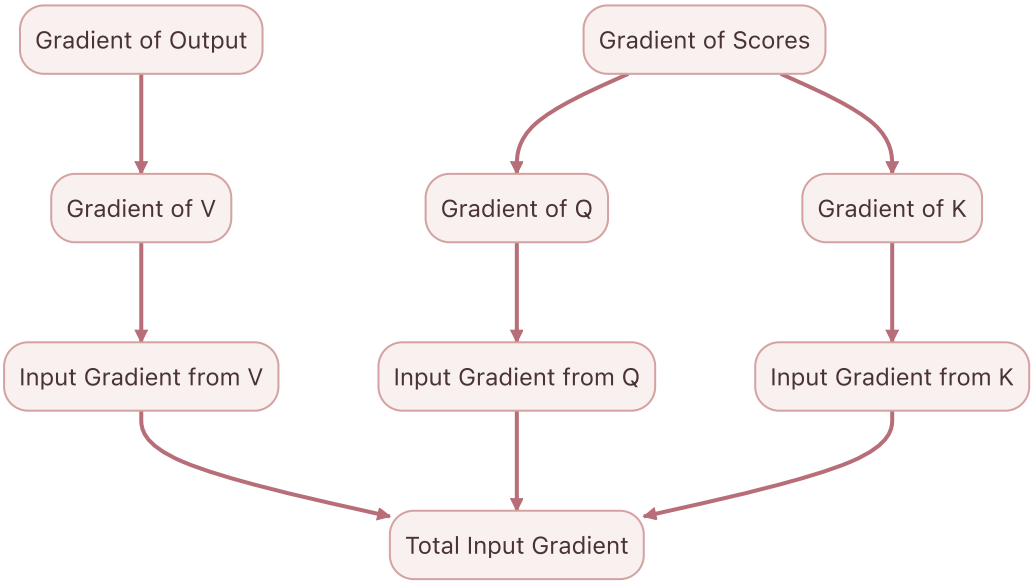
$$d_X_V = d_VW_V^T$$

$$d_X_Q = d_QW_Q^T$$

$$d_X_K = d_KW_K^T$$

Summing the three 4×6 branch contributions produces the total attention input gradient:

$$d_X_{\text{Total}} = d_X_V + d_X_Q + d_X_K$$



Evaluating the sum across all three branches yields the 4×6 input gradient matrix d_X:

$$d_X = \begin{bmatrix} 0.5536 & 0.1088 & -0.1536 & 0.0560 & 0.0463 & -0.3314 \\ 0.0465 & 0.1747 & -0.0985 & -0.1107 & 0.1189 & 0.0421 \\ 0.0159 & 0.1742 & -0.0346 & 0.2056 & -0.0277 & -0.0953 \\ 0.0348 & 0.0179 & -0.0020 & -0.0572 & 0.0338 & 0.0424 \end{bmatrix}$$

The backward traversal through self-attention is now complete. The mechanism has translated the upstream output and score gradients into explicit weight updates for W_Q , W_K , and W_V while assembling the unified 4×6 gradient d_X to continue down the residual stream. Chapter 24 traces this accumulated signal back to the initial token embedding lookup table to complete the training step.

Chapter 24: Updating the Embeddings and Conclusion

The backward pass now reaches the input boundary of the architecture. The error signal originated at the cross-entropy loss, traversed the unembedding projection W_U, branched across the Layer 2 MLP and attention blocks, and propagated through Layer 1. That accumulated gradient now arrives at the foundational token representation table: the embedding matrix W_E (denoted here as E).

The Residual Highway

During the forward pass, the residual stream operated as a shared memory bus. Initial token embeddings entered this highway, and each attention and MLP sub-layer added contextual updates back into the stream.

In the backward pass, the residual stream functions as a direct gradient highway. Differentiating each element-wise residual addition routes the upstream gradient equally to both the skip path and the sub-layer branch. The gradient arriving at the base of the residual stream therefore equals the sum of the direct skip-connection gradient and the gradients emerging from every attention and MLP block across both layers. This 4×6 matrix d_X encapsulates how each of the four input token vectors (`<BOS>`, `i`, `woke`, and `up`) must shift in $\mathbb{R}^{d_{\text{model}}}$ to reduce the sequence prediction loss:

$$d_X = \begin{bmatrix} -0.0036 & 0.1565 & -0.2620 & 0.0822 & 0.0087 & -0.0299 \\ 0.0092 & -0.1988 & -0.0220 & 0.0357 & 0.1478 & -0.0518 \\ -0.0808 & -0.0502 & 0.0915 & 0.0329 & -0.0530 & 0.0513 \\ 0.0097 & 0.0969 & -0.0702 & -0.0328 & -0.0392 & -0.1464 \end{bmatrix}$$

Routing Gradients to the Vocabulary Space

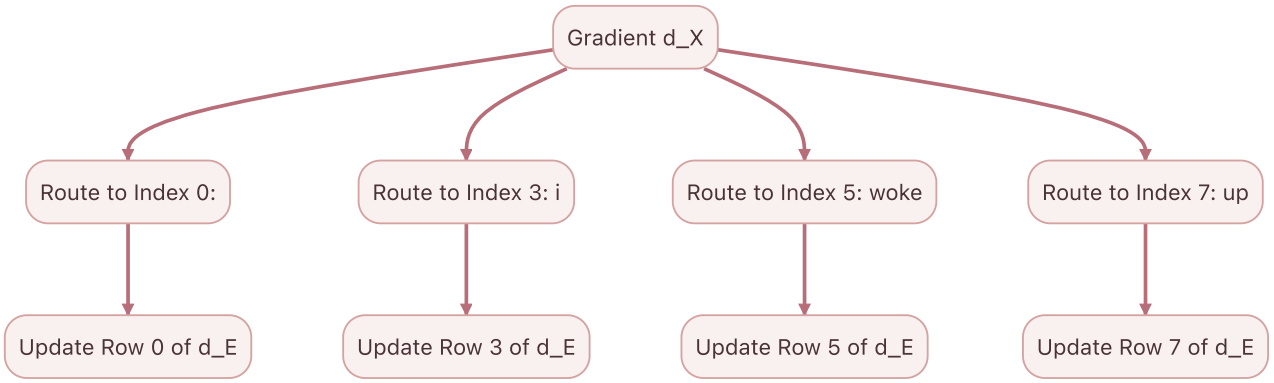
During the forward pass (Chapter 1), the 4×6 input matrix X was constructed by multiplying the 4×12 one-hot token matrix $X_{\text{one-hot}}$ by the 12×6 embedding matrix E:

$$X = X_{\text{one-hot}} E$$

Differentiating this lookup with respect to E multiplies the transposed one-hot matrix $X_{\text{one-hot}}^T$ (12×4) by the 4×6 gradient matrix d_X:

$$d_E = X_{\text{one-hot}}^T d_X$$

Multiplying by $X^T_{\text{one-hot}}$ scatters each row of d_X back into the vocabulary row index that was selected during the forward pass. Mapping the prompt `<BOS>` `i` `woke` `up` to vocabulary indices `[0, 3, 5, 7]` causes the 12×6 embedding gradient matrix d_E to accumulate rows `0, 1, 2, 3` of d_X into vocabulary rows `0, 3, 5, 7`, while the eight unused vocabulary rows receive a gradient of zero:



The Optimizer Update

With d_E assembled alongside the weight gradients for every attention and MLP projection matrix across the network, the optimizer executes the parameter update step.

Production training pipelines typically apply adaptive optimizers such as AdamW, which normalize parameter updates using running estimates of first and second gradient moments alongside decoupled weight decay. Stochastic Gradient Descent (SGD) illustrates the core geometric update rule using a scalar learning rate α :

$$E_{\text{new}} = E_{\text{old}} - \alpha \cdot d_E$$

Applying a learning rate of $\alpha = 0.01$ to Row 0 (`<BOS>` , where $d_E[0] = d_X[0]$ equals `[-0.0036, 0.1565, -0.2620, 0.0822, 0.0087, -0.0299]`) subtracts $0.01 \cdot d_X[0]$ from the initial embedding coordinates. Starting from a four-decimal Gaussian initialization $E_{\text{old}}[0]$ (or equivalently from Chapter 1's hand-crafted row $W_E[0]$ of `[0.1000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000]` , which updates to `[0.1000, -0.0016, 0.0026, -0.0008, -0.0001, 0.0003]`), the update shifts the six coordinates in the direction that lowers cross-entropy loss:

$$E_{\text{old}}[0] = [0.4967 \quad -0.1383 \quad 0.6477 \quad 1.5230 \quad -0.2342 \quad -0.2341]$$

$$E_{\text{new}}[0] = [0.4967 \quad -0.1399 \quad 0.6503 \quad 1.5222 \quad -0.2343 \quad -0.2338]$$

(Evaluating the subtraction on unrounded floating-point draws before four-decimal rounding yields `-0.1398` in column 2 and `-0.2342` in column 5.)

Subtracting $\alpha \cdot d_E$ translates the active token vectors (`<BOS>` , `i` , `woke` , and `up`) within the $d_{\text{model}} = 6$ embedding space. On the next forward pass, the updated coordinates for `woke` and `up` enter the residual stream pre-aligned with the attention and MLP subspaces that increase the output probability of the target token `late` .

Conclusion

This parameter update completes a full forward and backward training cycle of the Decoder-only Transformer. The forward pass mapped discrete vocabulary indices into continuous vectors in $\mathbb{R}^{d_{\text{model}}}$, injected sinusoidal positional signatures, routed asymmetric context across three parallel attention heads per layer, stabilized activations via Layer Normalization, and extracted higher-order features through two Key-Value MLP blocks before projecting back to vocabulary logits.

The backward pass reversed every stage of that computational graph via the chain rule. Starting from the linear cancellation $P - Y$ at the cross-entropy output boundary, gradients flowed backward through the unembedding matrix, split across residual additions, traversed the MLP projections and ReLU gates, navigated the coupled Softmax Jacobian and causal mask, and updated every projection weight down to the initial embedding lookup table.

Viewed through explicit matrix arithmetic, the Transformer is not an opaque black box. Every forward prediction and backward weight update reduces to deterministic linear projections, dot-product alignments, and coordinate translations in vector space.